UNIVERSITATEA DIN CRAIOVA
FACULTATEA DE AUTOMATICĂ, CALCULATOARE ȘI ELECTRONICĂ

DEPARTAMENTUL DE CALCULATOARE ȘI TEHNOLOGIA INFORMAȚIEI

PROIECT DE DIPLOMĂ

Mazilu Mihai

COORDONATOR ȘTIINȚIFIC

Profesor dr. ing. Mihai Mocanu

CRAIOVA, ROMÂNIA

Iulie 2026

UNIVERSITATEA DIN CRAIOVA
FACULTATEA DE AUTOMATICĂ, CALCULATOARE ȘI ELECTRONICĂ

DEPARTAMENTUL DE CALCULATOARE ȘI TEHNOLOGIA INFORMAȚIEI

AcademicNode

Mazilu Mihai

COORDONATOR ȘTIINȚIFIC

Profesor dr. ing. Mihai Mocanu

CRAIOVA, ROMÂNIA

Iulie 2026

DECLARAȚIA DE ORIGINALITATE

Subsemnatul Mazilu Mihai, student la specializarea Calculatoare în Limba Engleză din cadrul Facultății de Automatică, Calculatoare și Electronică a Universității din Craiova, certific prin prezenta că am luat la cunoştinţă de cele prezentate mai jos şi că îmi asum, în acest context originalitatea proiectului meu de licenţă:

- cu titlul Full stack web application with the involvement of an AI assistant, for academic use,
- coordonată de Profesor dr. ing. Mihai Mocanu
- prezentată în sesiunea Iulie 2026.

La elaborarea proiectului de licenţă, se consideră plagiat una dintre următoarele acţiuni:

1. reproducerea exactă a cuvintelor unui alt autor, dintr-o altă lucrare, în limba română sau prin traducere dintr-o altă limbă, dacă se omit ghilimele şi referinţa precisă,
2. redarea cu alte cuvinte, reformularea prin cuvinte proprii sau rezumarea ideilor din alte lucrări, dacă nu se indică sursa bibliografică,
3. prezentarea unor date experimentale obţinute sau a unor aplicaţii realizate de alţi autori fără menţionarea corectă a acestor surse,
4. însuşirea totală sau parţială a unei lucrări în care regulile de mai sus sunt respectate, dar care are alt autor.

Pentru evitarea acestor situaţii neplăcute se recomandă:

- plasarea între ghilimele a citatelor directe şi indicarea referinţei într-o listă corespunzătoare la sfărşitul lucrării,
- indicarea în text a reformulării unei idei, opinii sau teorii şi corespunzător în lista de referinţe a sursei originale de la care s-a făcut preluarea,
- precizarea sursei de la care s-au preluat date experimentale, descrieri tehnice, figuri, imagini, statistici, tabele et caetera,
- precizarea referinţelor poate fi omisă dacă se folosesc informaţii sau teorii arhicunoscute, a căror paternitate este unanim cunoscută și acceptată.

Data, 28.05.2026 Semnătura candidatului, 

UNIVERSITATEA DIN CRAIOVA
Facultatea de Automatică, Calculatoare şi Electronică

Departamentul de Calculatoare și Tehnologia Informației

Aprobat la data de
01.12.2025
Şef de departament,
Sef Lucrari dr. ing.
Nicolae ENESCU

PROIECTUL DE DIPLOMĂ

Numele și prenumele studentului/-ei:	Mihai Mazilu
Enunțul temei:	Full stack web application using Angular and .Net, databases, with the involvement of an AI assistant, for academic use at departmental level (administrative/teaching/research)
Datele de pornire:	Decembrie 2025
Conținutul proiectului:	Capitolul 1: Introducere și Motivație Prezentarea situației actuale a ecosistemului educațional, articularea problemei fragmentării instrumentelor digitale și enunțarea obiectivului platformei AcademicNode ca furnizând o soluție centralizată și sigură. Capitolul 2: Tehnologiile și Instrumentele Utilizate Framework-uri Back-end (.NET 8, SQL Server, EF Core) și tehnologii front-end (Angular 21, PrimeNG, RxJS) împreună cu tehnologii de infrastructură (Docker, Tailscale) pentru evaluarea tehnică a stack-ului utilizat. Capitolul 3: Arhitectura Sistemului Structura aplicației care detaliază arhitectura decuplată Client-Server, modelul relațional al bazei de date și schema de implementare fizică și logică izolată (Zero-Trust) pe un server NAS privat. Capitolul 4: Implementare și Securitate Descrierea codului și principalele caracteristici (flux CRUD pentru profile, managementul postărilor, integrarea asistentului AI Gemini), precum și mecanismele de securitate implimentate, inclusiv autentificarea JWT.

	Capitolul 5: Concluzii și Dezvoltări Ulterioare Sumarizarea contribuțiilor personale și tehnice aduse prin realizarea proiectului, alături de analizarea limitărilor curente și conturarea unui plan (Roadmap) pentru viitoarele funcționalități și scalări.
Material grafic obligatoriu:	Diagrame arhitecturale și de rețea (ex. arhitectura decuplată Client-Server, topologia de deployment Zero-Trust pe server NAS). Diagrame logice și de securitate (ex. fluxul de autentificare cu token-uri JWT, generarea bazei de date Code-First). Seturi de imagini din interfața aplicației (ex. formularele de creare postări cu insigne dinamice pentru profesori, ferestrele de editare a profilului, asistentul AI Gemini). Grafice conceptuale și de planificare (ex. comparația vizuală între ecosistemul academic tradițional și AcademicNode, Roadmap-ul pentru dezvoltări viitoare).
Consultații:	Periodice
Conducătorul științific (titlul, nume și prenume, semnătura):	Profesor dr. ing. Mihai Mocanu
Data eliberării temei:	15.11.2025
Termenul estimat de predare a proiectului:	31.05.2026
Data predării proiectului de către student și semnătura acestuia:	30.05.2026

UNIVERSITATEA DIN CRAIOVA
Facultatea de Automatică, Calculatoare şi Electronică

Departamentul de Calculatoare și Tehnologia Informației

REFERATUL CONDUCĂTORULUI ȘTIINȚIFIC

Numele și prenumele candidatului/-ei:	Mazilu Mihai
Specializarea:	Calculatoare în Limba Engleză
Titlul proiectului:	AcademicNode
Locația în care s-a realizat practica de documentare (se bifează una sau mai multe din opțiunile din dreapta):	În facultate ☐ În producție ☐ În cercetare ☐ Altă locație: [*se detaliază*]

În urma analizei lucrării candidatului au fost constatate următoarele:

Nivelul documentării		Insuficient ☐	Satisfăcător ☐	Bine ☐	Foarte bine ☐
Tipul proiectului		Cercetare ☐	Proiectare ☐	Realizare practică ☐	Altul [*se detaliază*]
Aparatul matematic utilizat		Simplu ☐	Mediu ☐	Complex ☐	Absent ☐
Utilitate		Contract de cercetare ☐	Cercetare internă ☐	Utilare ☐	Altul [*se detaliază*]
Redactarea lucrării		Insuficient ☐	Satisfăcător ☐	Bine ☐	Foarte bine ☐
Partea grafică, desene		Insuficientă ☐	Satisfăcătoare ☐	Bună ☐	Foarte bună ☐
Realizarea practică	Contribuția autorului	Insuficientă ☐	Satisfăcătoare ☐	Mare ☐	Foarte mare ☐
	Complexitatea temei	Simplă ☐	Medie ☐	Mare ☐	Complexă ☐
	Analiza cerințelor	Insuficient ☐	Satisfăcător ☐	Bine ☐	Foarte bine ☐
	Arhitectura	Simplă ☐	Medie ☐	Mare ☐	Complexă ☐

	Întocmirea specificațiilor funcționale	Insuficientă ☐	Satisfăcătoare ☐	Bună ☐	Foarte bună ☐
	Implementarea	Insuficientă ☐	Satisfăcătoare ☐	Bună ☐	Foarte bună ☐
	Testarea	Insuficientă ☐	Satisfăcătoare ☐	Bună ☐	Foarte bună ☐
	Funcționarea	Da ☐	Parțială ☐	Nu ☐	
Rezultate experimentale		Experiment propriu ☐		Preluare din bibliografie ☐	
Bibliografie		Cărți	Reviste	Articole	Referințe web
Comentarii și observații					

În concluzie, se propune:

ADMITEREA PROIECTULUI	**RESPINGEREA PROIECTULUI**
☐	☐

Data, Semnătura conducătorului științific,

REZUMATUL PROIECTULUI

AcademicNode este o platformă hibridă care reunește caracteristicile unui sistem de gestionare a învățării și elementele interactive ale rețelelor sociale profesionale. Scopul său principal este de a oferi studenților și profesorilor un spațiu digital pentru a lucra împreună, a partaja resurse academice precum documente PDF și a crea profiluri profesionale detaliate.

Am construit platforma cu o configurație client-server separată. Interfața cu utilizatorul folosește Angular 21 pentru a crea o aplicație interactivă pe o singură pagină. Logica de business și datele sunt gestionate de o API RESTful în .NET 8, care se conectează la o bază de date SQL Server prin Entity Framework Core. Pentru securitate și gestionarea rolurilor precum Administrator, Profesor și Student, am folosit autentificarea fără stare cu token-uri JWT.

Una dintre cele mai mari provocări în timpul dezvoltării a fost configurarea relațiilor recursive de tip many-to-many pentru sistemul de Urmăritori și Aprecieri, prevenind în același timp ștergerile în cascadă nedorite din baza de date. O altă provocare a fost asigurarea accesibilității securizate a aplicației de pe un server NAS privat. Pentru a rezolva acest lucru, am folosit Docker pentru a containeriza aplicația și am configurat o rețea Zero-Trust cu Tailscale VPN, ceea ce a eliminat riscurile deschiderii porturilor publice. Am livrat un ecosistem software funcțional, scalabil și sigur. Acest proiect m-a ajutat să-mi aprofundez înțelegerea arhitecturii software moderne, a bazelor de date relaționale și a practicilor DevOps necesare pentru lansarea aplicațiilor în producție.

Cuvinte cheie: Rețea socială academică, Aplicație cu o singură pagină, Angular 21, API web .NET 8, Entity Framework Core, Arhitectură Zero-Trust, Docker.

PROJECT SUMMARY

AcademicNode is a hybrid platform that brings together the features of a learning management system and the interactive elements of professional social networks. Its main purpose is to give students and teachers a digital space to work together, share academic resources like PDF documents, and create detailed professional profiles.

I built the platform with a separate client-server setup. The user interface uses Angular 21 to create an interactive single-page application. The business logic and data are handled by a RESTful API in .NET 8, which connects to a SQL Server database through Entity Framework Core. For security and managing roles like Administrator, Teacher, and Student, I used stateless authentication with JWT tokens.

One of the biggest challenges during development was setting up the recursive many-to-many relationships for the Followers and Likes system, while also preventing unwanted cascading deletes in the database. Another challenge was making the application securely accessible from a private NAS server. To solve this, I used Docker to containerize the app and set up a Zero-Trust network with Tailscale VPN, which removed the risks of opening public ports.

I delivered a software ecosystem that is functional, scalable, and secure. This project helped me deepen my understanding of modern software architecture, relational databases, and the DevOps practices needed to launch applications in production.

Keywords: Academic Social Network, Single Page Application, Angular 21, .NET 8 Web API, Entity Framework Core, Zero-Trust Architecture, Docker.

In the era of accelerated digitalization and profound transformation of higher education institutions, efficient management of academic flows and secure interaction between students and faculty are major challenges for current IT infrastructures. This work, structured around the development and implementation of the AcademicNode platform, addresses these needs by proposing a modern, scalable, and highly secure software solution.

The author's interest in software engineering and cybersecurity is clearly reflected in the complex architecture of the proposed application. Technologically, the project demonstrates remarkable maturity by using a modern hybrid ecosystem: a dynamic interface developed in Angular and PrimeNG, a robust backend based on the .NET 8 Web API framework, and a database management system implemented in SQL Server via Entity Framework Core.

This thesis gains special value not only from developing Fullstack software components but also from a rigorous approach to network infrastructure and production deployment. In a digital landscape marked by constant cyber threats, the author's decision to isolate the application using a Zero-Trust architecture implemented with Tailscale (Mesh VPN) and containerized with Docker shows a deep understanding of modern SecOps principles. Moreover, validating the platform's extensibility through a Hybrid-Cloud Proof of Concept in Microsoft Azure underscores the project's applied and enterprise-oriented nature.

During development, the author demonstrated remarkable qualities as a researcher and engineer: rigor in designing the database architecture, attention to detail in optimizing the user experience (UI/UX), and excellent ability to solve complex technical problems during deployment. Rigorous testing of endpoints and practical demonstration of the system's operation under network isolation provide full credibility to the results.

I believe this work goes beyond the standard level of a diploma project, constituting a valuable contribution to the field of secure web applications. I recommend this thesis with full confidence to the evaluation committee, being convinced that the demonstrated technical solutions pave the way to a successful career in the information technology industry.

Scientific Coordinator,

Profesor dr. ing. Mihai Mocanu

Contents

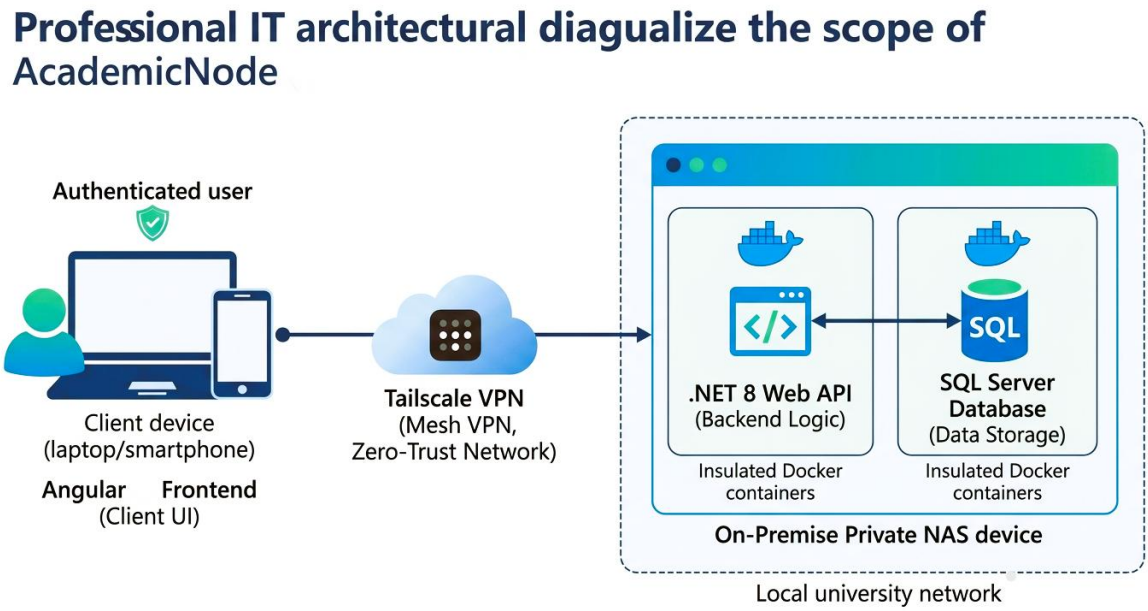
1 INTRODUCTION

1.1 Scope

This thesis presents the design, development, and implementation of **AcademicNode**, a modern academic networking platform. The project creates a digital ecosystem for students, professors, and university administrators.

Its goal is to connect the features of traditional Learning Management Systems (LMS) with those of professional social networks. The application supports real-time communication, centralized sharing of academic resources such as PDF documents, and professional profiles.

It is meant to be a single hub for academic collaboration. This thesis also explains and justifies the main software engineering choices, including a decoupled Client-Server architecture using Angular 21 and .NET 8 Web API, and a Zero-Trust deployment strategy on a private NAS device with Tailscale VPN.



1.2 Motivation

I chose the theme "AcademicNode" because I saw a need in the academic world for a platform that brings together social interaction and educational resources in a secure and efficient way. Most current learning management systems are rigid and focus mainly on administrative tasks, missing the social network features that help students and teachers collaborate more easily. I was also personally interested in using new technologies.

For this project, I used Angular 21 for the user interface, .NET 8 for the backend, and Microsoft SQL Server for data management. This combination allowed me to build an application that meets industry standards. I was especially motivated by the challenges of infrastructure and security. By hosting the app on a private NAS server, using Docker containers to separate services, and setting up a Zero-Trust network with Tailscale VPN, I focused on keeping academic data secure and independent from public cloud providers.

This project not only meets practical needs like sharing PDF courses, tracking progress, and offering an integrated chat, but it also gave me a chance to study modern Single Page Application architectures and DevOps practices in depth.

1.3 Actual Context

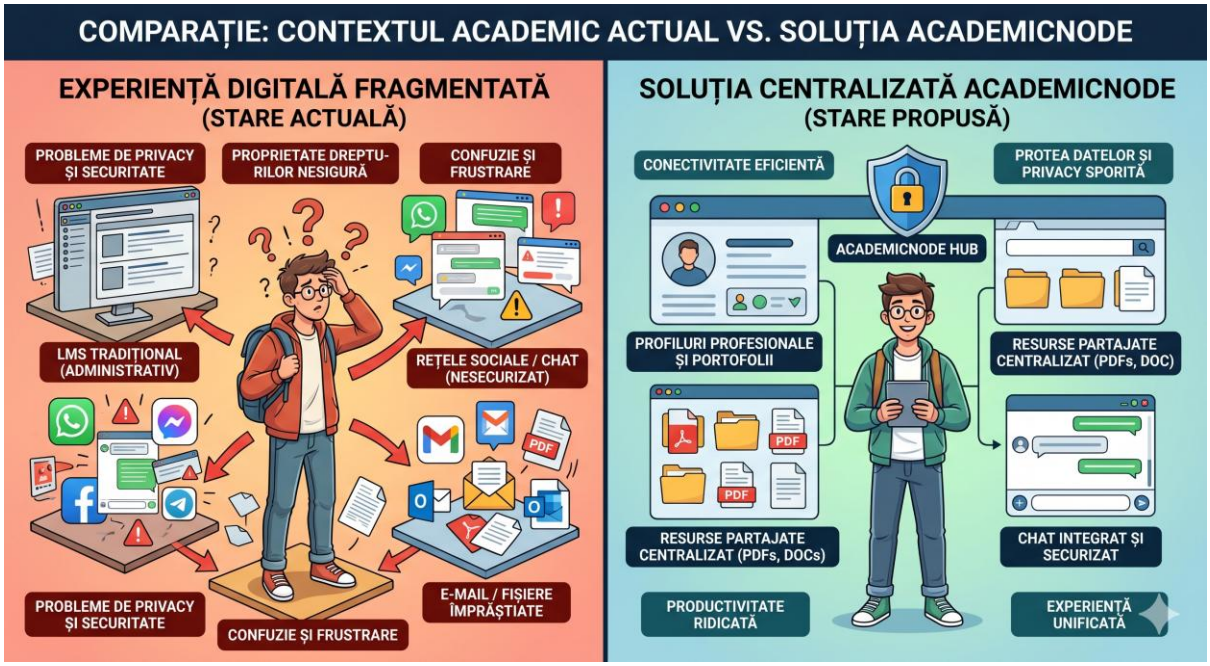
Today, academic institutions are becoming more digital at a rapid pace. Still, most software used in higher education is fragmented. Universities often depend on traditional Learning Management Systems (LMS) that mainly handle administrative tasks, course management, and grading. While these systems work well for administration, they are usually rigid and do not offer the user-friendly, community features needed for easy academic collaboration and networking.

As a result, students and staff often move their conversations to general social media and messaging apps. This leads to a scattered digital experience, with important academic resources and discussions spread across unsecured and informal channels. This situation

weakens academic focus and raises real concerns about data privacy and unclear professional boundaries.

From a software engineering point of view, the industry now favors decoupled architectures, containerization, and strong data security. Building large, single-piece applications on exposed servers is no longer best practice. Today's systems need to be highly available, secure, and easy to update and deploy.

Given these challenges, building AcademicNode makes sense right now. It meets the need for a single, secure, and modern academic networking platform. AcademicNode offers a system for sharing resources and building professional profiles, and uses a Zero-Trust network setup with Tailscale and Docker on a NAS device. This approach solves the problems of current educational tools and matches the latest trends in software and cybersecurity.



2 Technologies and Tools Used

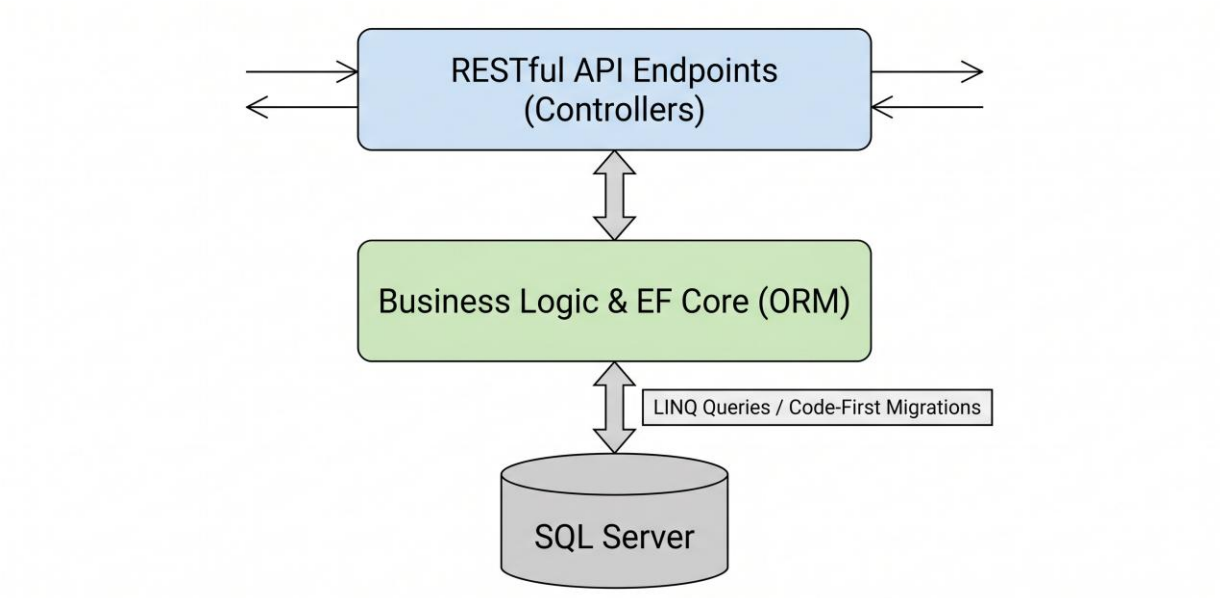
Building AcademicNode required choosing technologies that focus on performance, security, and developer productivity. The project uses a modern, decoupled architecture with industry-standard frameworks to support scalability and deliver a strong user experience.

2.1. Back-End Technologies (.NET 8, SQL Server, EF Core)

The server-side infrastructure is built on Microsoft technologies, giving the platform a stable and high-performance base for business logic and data storage using the .NET 8 framework. This version was selected for its cross-platform capabilities, superior execution speed, and comprehensive support for building RESTful services. The use of asynchronous programming models allows the system to handle concurrent requests efficiently, which is vital for a collaborative platform.

Microsoft SQL Server: The system uses Microsoft SQL Server to store data. This database offers strong reliability and advanced query features. It works well for handling complex data structures like user roles and the network of followers on the platform.

Entity Framework Core (EF Core): EF Core connects the C# code to the database as an Object-Relational Mapper (ORM). It lets developers use LINQ for queries, which keeps data types safe and helps prevent SQL injection. The project uses a "Code-First" approach, so the database can change easily with automated migrations.

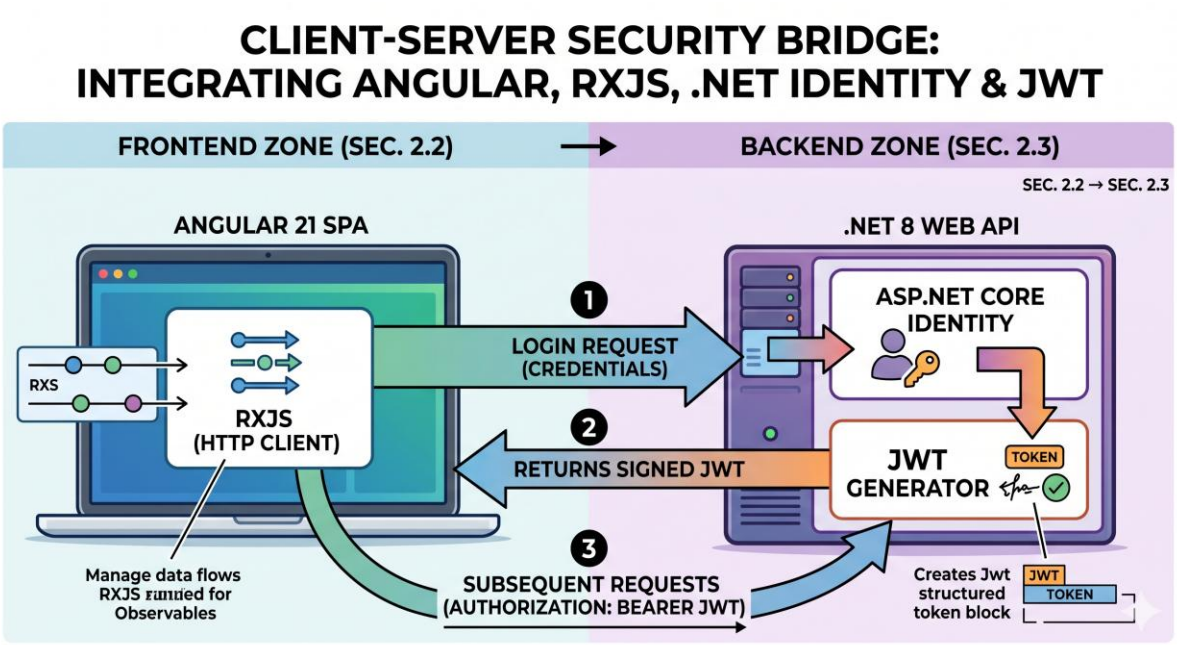


2.2. Front-End Technologies (Angular 21, PrimeNG, RxJS)

The client side is built as a reactive Single Page Application (SPA) to provide a smooth and interactive user interface.

Angular 21 and TypeScript: The user interface uses Angular 21, which offers a component-based structure and strong dependency injection. TypeScript is used everywhere to enforce strict typing, reduce errors, and make the code easier to maintain.

PrimeNG 21: The project uses the PrimeNG library for a professional and consistent look. It offers many ready-made, responsive UI components like dynamic tables, interactive modals, and data visualization tools, which helped speed up development and improve the user experience. Reactive Extensions for JavaScript): All asynchronous data streams, including API requests and real-time state updates, are managed using RxJS observables. This ensures that the application remains responsive, allowing for instant UI updates when users interact with posts or follow peers.

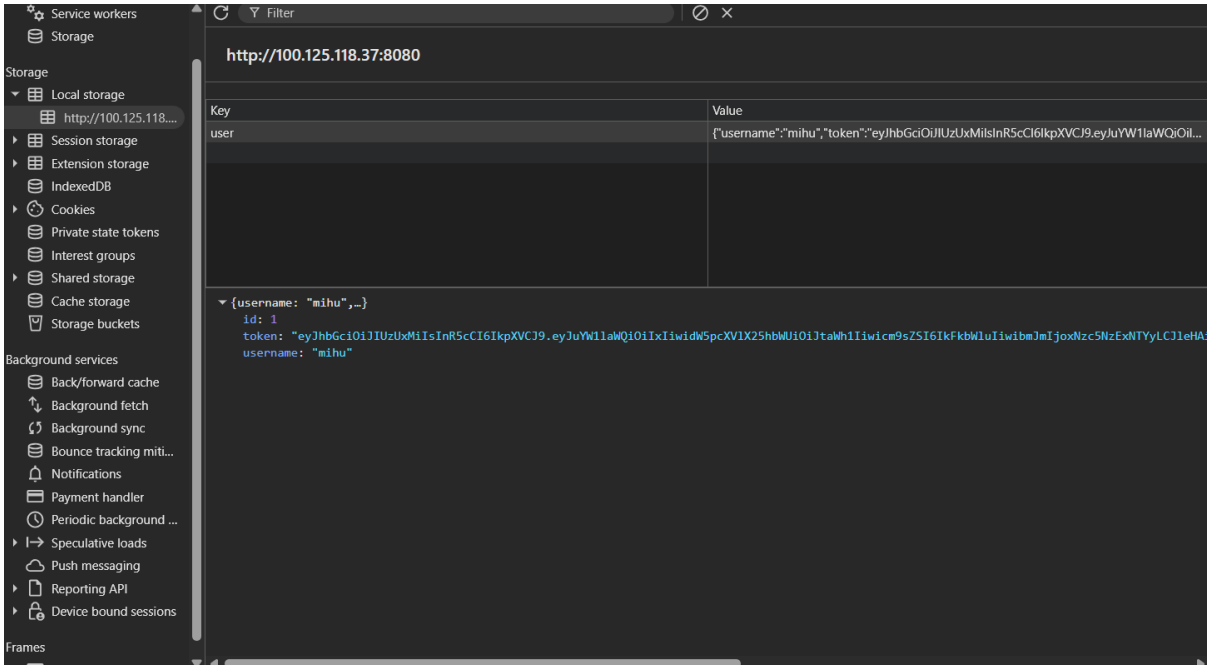


2.3. Security and Identity Management (Identity, JWT)

Security uses several layers to protect user credentials and sensitive academic data.

ASP.NET Core Identity: This framework handles authentication and authorization. It manages password hashing, user registration, and roles like Administrator, Professor, and Student. The system uses a custom identity store to meet AcademicNode's specific needs.

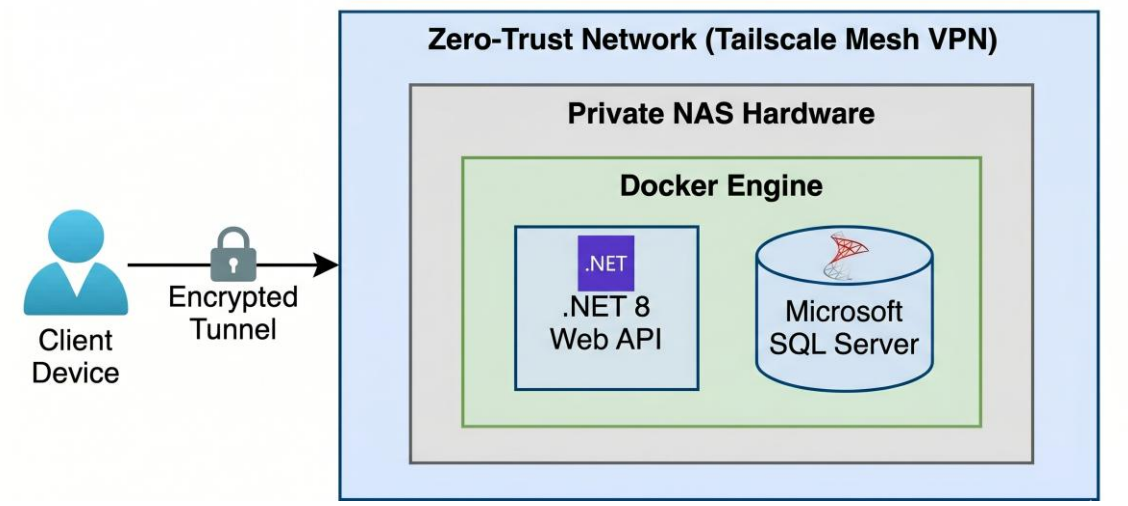
JSON Web Tokens (JWT): To achieve a stateless and scalable authentication model, JSON Web Tokens (JWT): are used for stateless and scalable authentication. After a user logs in, the server gives them a signed token, which the client sends with future requests. This removes the need for server-side session storage, improving performance and scalability for *AcademicNode* emphasizes isolation, portability, and secure remote access.



2.4. Infrastructure and Deployment Tools

Docker Containerization: The backend, including the .NET API and SQL Server, runs in Docker containers. This keeps the environment consistent across different hardware, making deployment easier and avoiding issues where code works on one machine but not another.

Private NAS Hosting: The app is hosted on a private Network Attached Storage (NAS) device. This setup gives full control over hardware and local data management, which is important for storing lots of user-uploaded academic files and media.



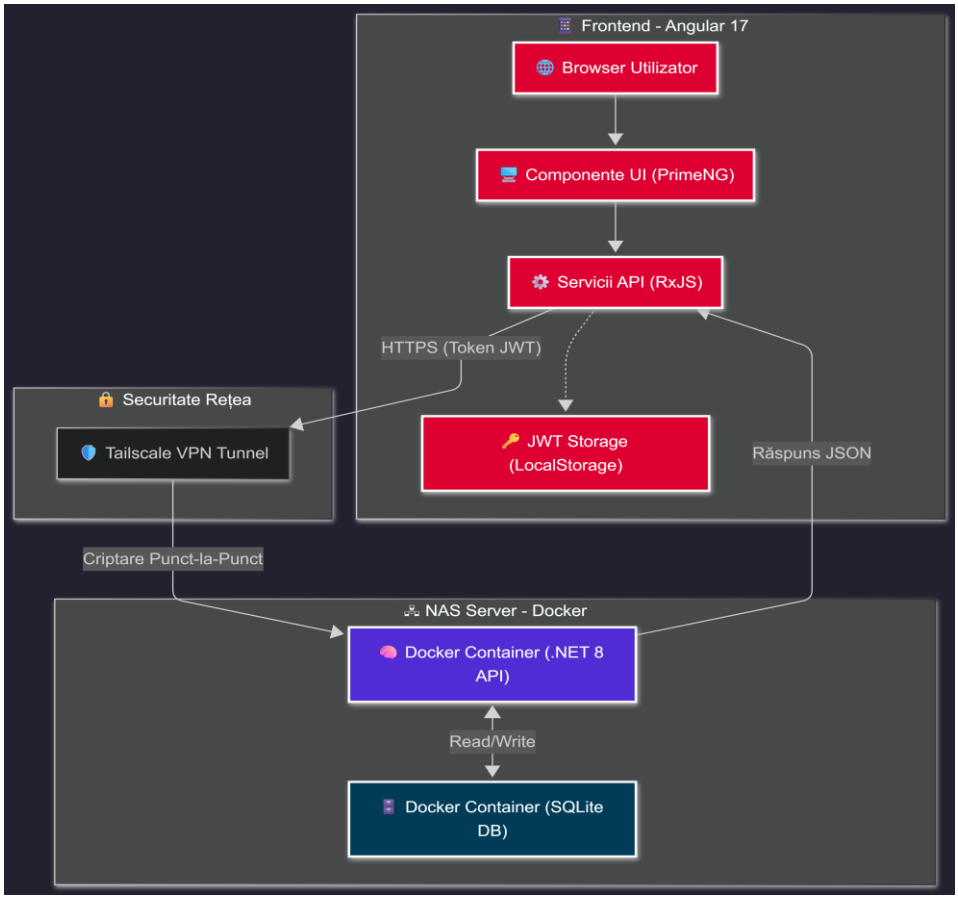
Tailscale VPN (Zero-Trust Architecture): Tailscale is used to provide secure remote access without public port forwarding. It creates an encrypted VPN tunnel, connecting approved client devices and the server on a secure virtual network. This keeps the API hidden from the public internet and only accessible through authenticated VPN nodes.

3. System Architecture and Design

This chapter explains how the AcademicNode platform is built and organized. The system uses a multi-tier, decoupled design to keep different parts separate, protect data, and maintain strong security.

3.1. Overall Application Architecture (Client-Server)

The platform uses a modern client-server setup with a decoupled structure. This means the user interface and the core business logic are kept separate, so each part can grow and change on its own.



The front-end is built with Angular 21 and works as a single-page application. It controls the user interface, manages changes in real time using RxJS, and talks to the backend only through secure RESTful API calls.

The back-end runs on .NET 8 Web API and acts as the main controller. It handles incoming HTTP requests, checks business rules like follower relationships or file upload limits, and manages authentication with JSON Web Tokens (JWT).

The data layer uses Microsoft SQL Server to store user profiles, social activity, and academic records for the long term. The API connects to the database using Entity Framework Core.

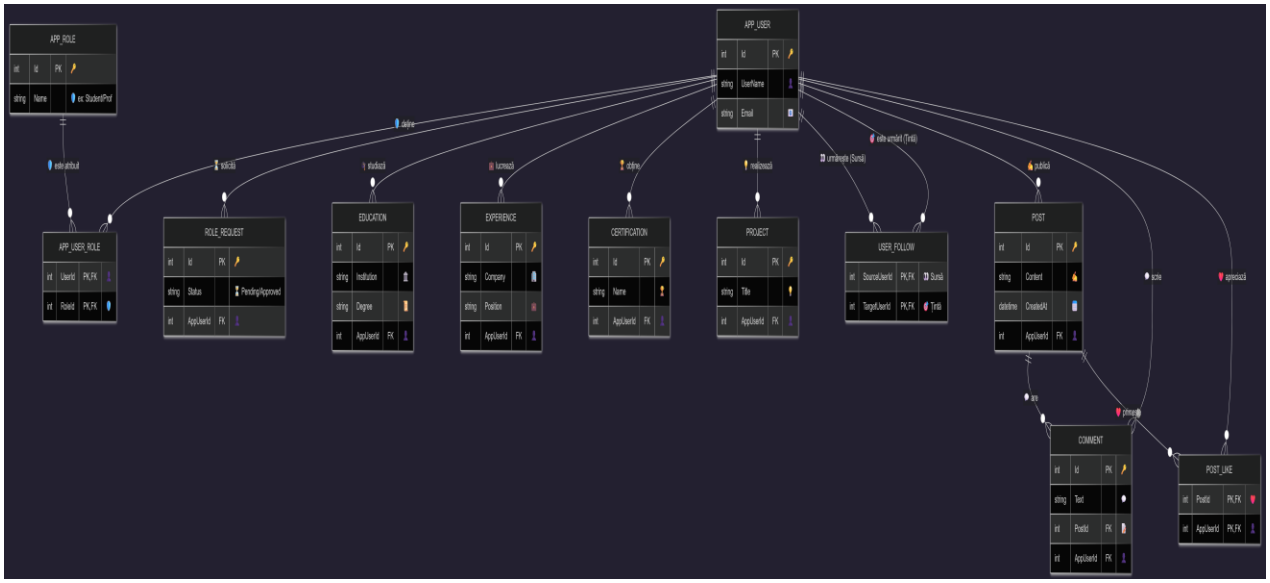
3.2. Database Modeling (ERD and Relationships)

The database is set up to handle complex social features and detailed user profiles. It uses a relational model to keep data consistent.

For identity and access, the system uses ASP.NET Core Identity with AppUser and AppRole entities. The AppUserRole table allows users to have more than one role, such as Student, Professor, and Admin.

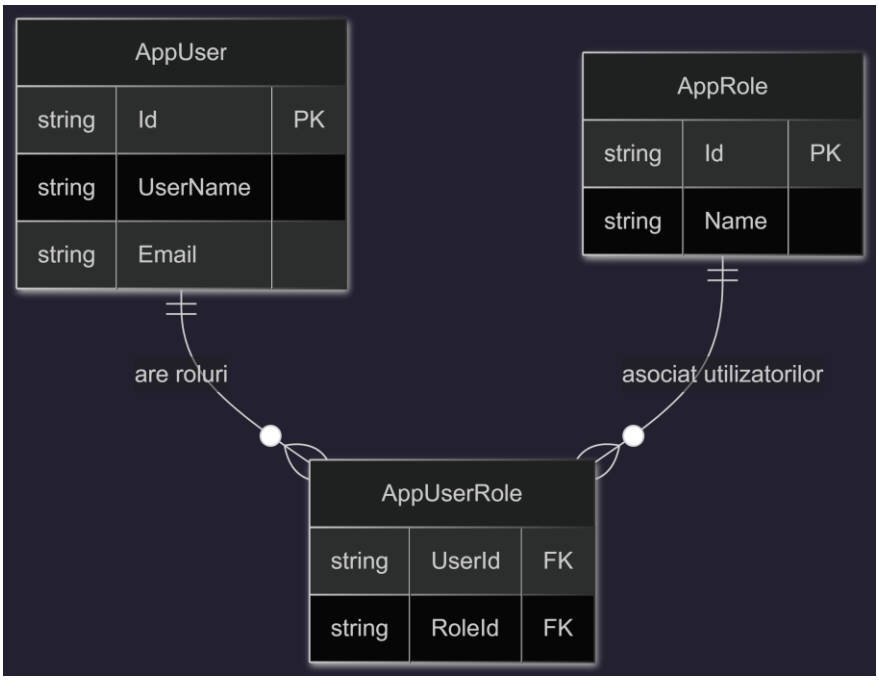
Professional Profiling: The user profile includes details like education, experience, certifications, and projects, each linked in a one-to-many relationship. This setup gives a detailed view of each user's academic and work history. It implements a self-referencing Many-to-Many relationship on the AppUser table. It tracks SourceUserId (the follower) and TargetUserId (the followed), enabling the social networking core of the platform.

Posts can include text or media, such as PDF files. Users can like or comment on posts. To avoid problems in the database, some relationships use DeleteBehavior.NoAction so that deleting a user or post does not cause unexpected errors.

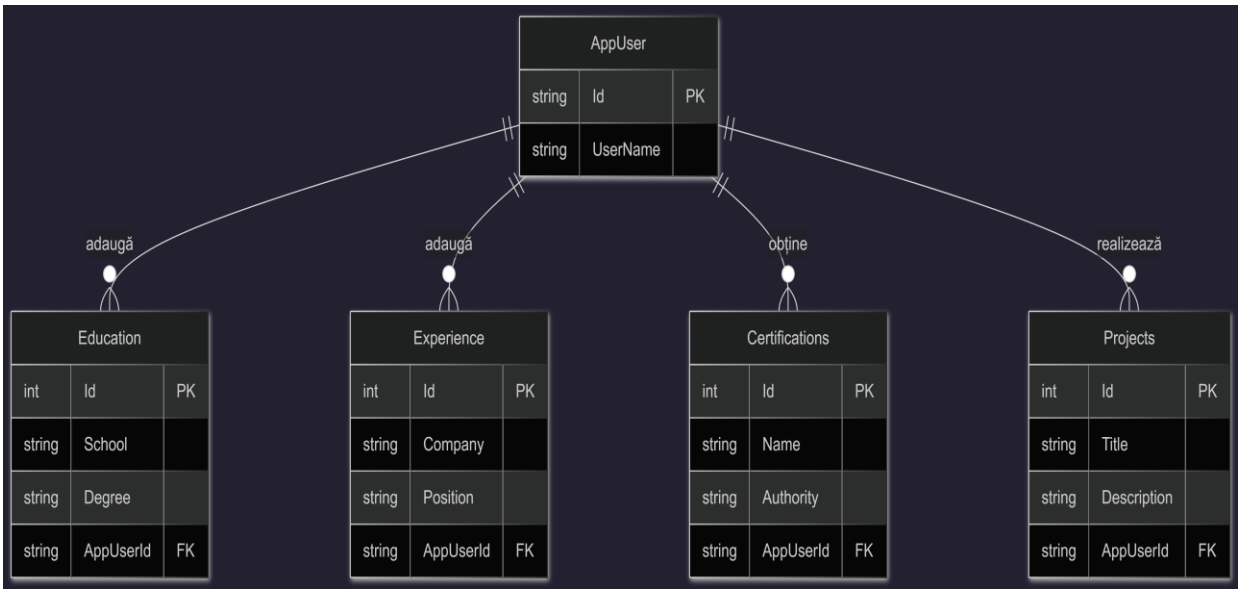


The Entity Relationship Diagram is divided in 3 parts:

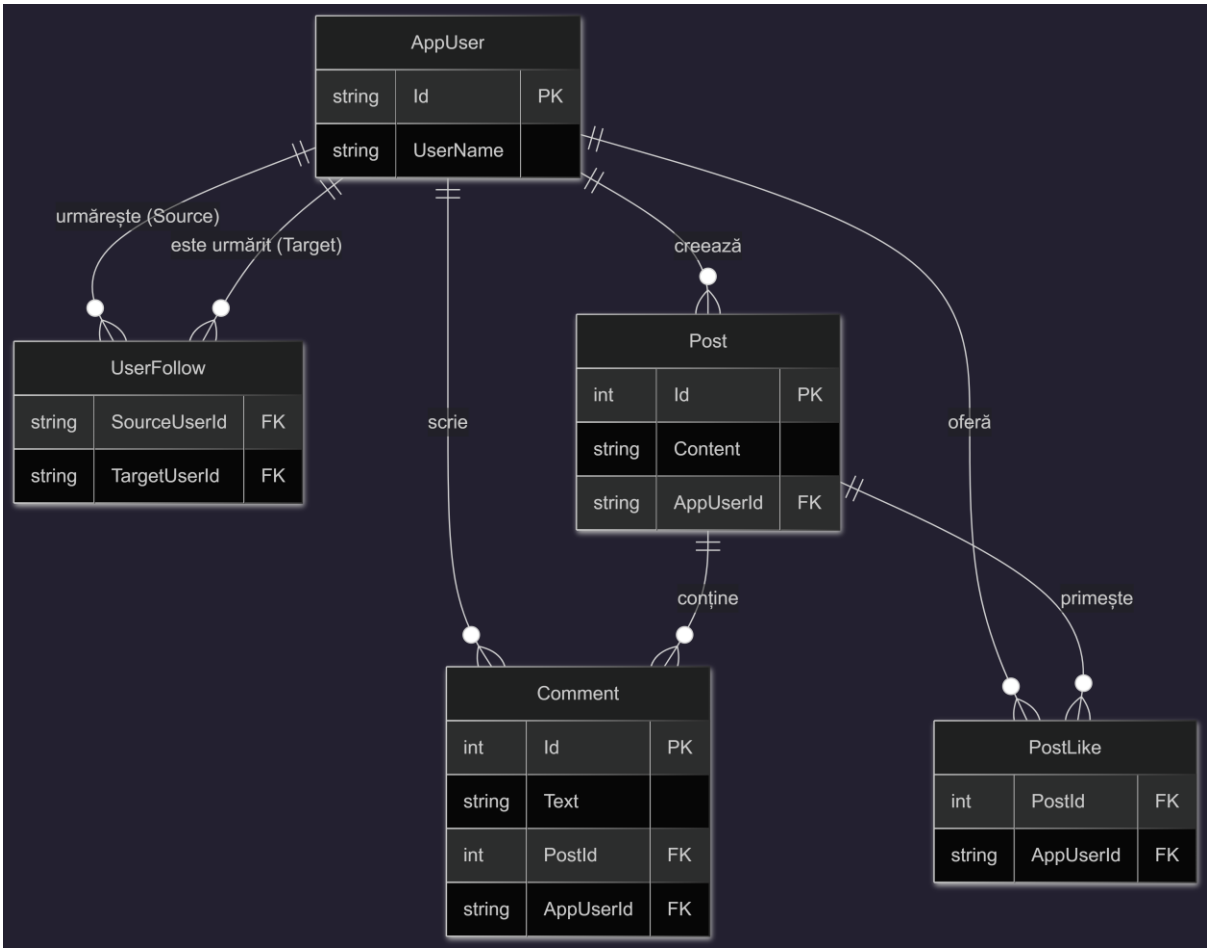
1) **Security and Identity Module (Identity & Access):**



2) **Academic Professional Profile Module (User Profile):**



3) Social & Interaction Module:

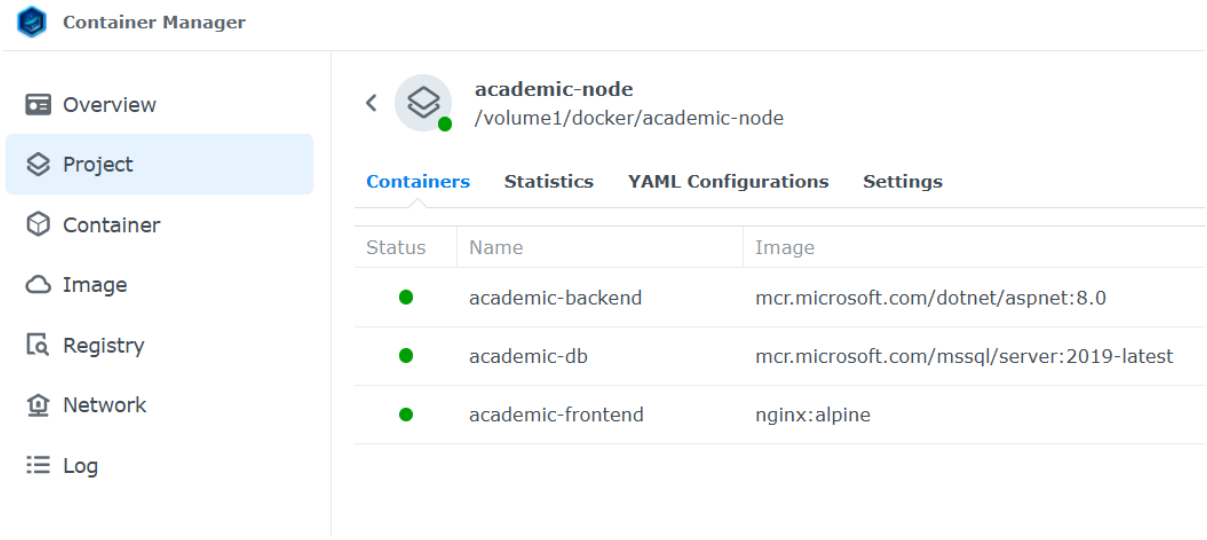


3.3. Network Topology and Infrastructure Security (Zero-Trust)

One of AcademicNode's main technical strengths is its deployment strategy, which keeps the system fully isolated from public internet risks.

The API and SQL Server run inside Docker containers. This keeps them separate from the host operating system, ensures they work the same way every time, and makes it easy to scale across NAS hardware.

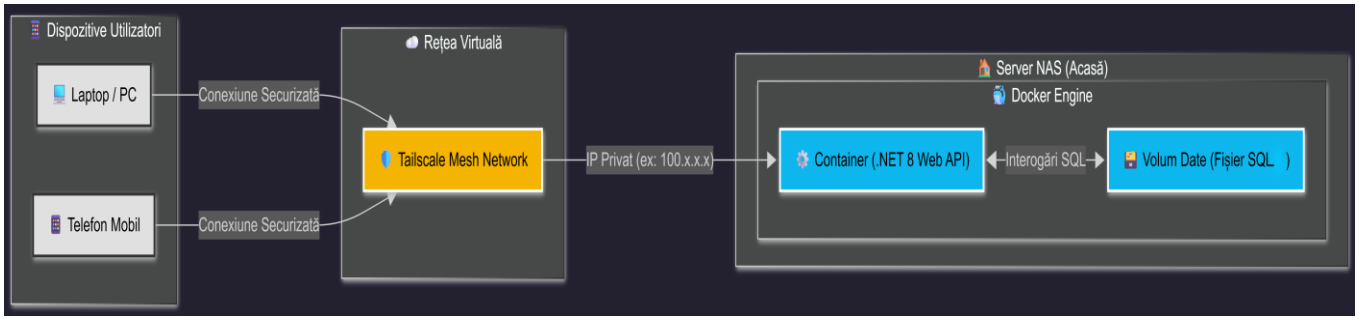
NAS-Based Hosting: By hosting the system on a private Network Hosting the system on a private NAS gives the platform has full control over where data is stored. This is important for handling academic documents and files uploaded by users. Risks of traditional Port Forwarding, the platform employs a Zero-Trust architecture via Tailscale. It creates an encrypted Mesh VPN tunnel (using the WireGuard protocol), assigning private IP addresses to authorized devices. Consequently, the API and database are invisible to the public internet and can only be accessed by authenticated nodes within the secure virtual perimeter.



3.4. Physical and Logical Deployment Architecture

To show how the platform works in a real-world setting, the deployment architecture was designed to highlight the system's physical and logical boundaries. The figure shows the UML Deployment Diagram for AcademicNode, outlining how data moves from users to the secure on-premise hardware.

The architecture is divided into three main operational zones. This setup keeps responsibilities separate and helps protect data.



The Client Devices Zone, or User Endpoints, is the outer layer of the system. It includes the laptops, PCs, and mobile devices used by students, professors, and administrators. These devices run the Angular-based Single Page Application in their web browsers. Users must be authenticated and authorized to access the platform.

The Zero-Trust Virtual Network, or Tailscale Mesh, serves as the main security layer. It replaces public web servers and open router ports. Client devices do not send HTTP requests over the public internet. Instead, they connect securely and with encryption to the Tailscale

Mesh Network. Each trusted device gets a private internal IP address, such as 100.x.x.x, creating a hidden Wide Area Network (WAN) using the WireGuard protocol.

The On-Premise Execution Environment (NAS Server): The On-Premise Execution Environment, or NAS Server, is the core of the platform. It runs on a private, home-based Network Attached Storage (NAS) server that hosts the Docker Engine. Within this secure setup, the containerized .NET 8 Web API handles incoming requests. The API uses a SQL case container (SQL Server) to store academic data safely using persistent Docker volumes. It protects the data even if containers are restarted or replaced.

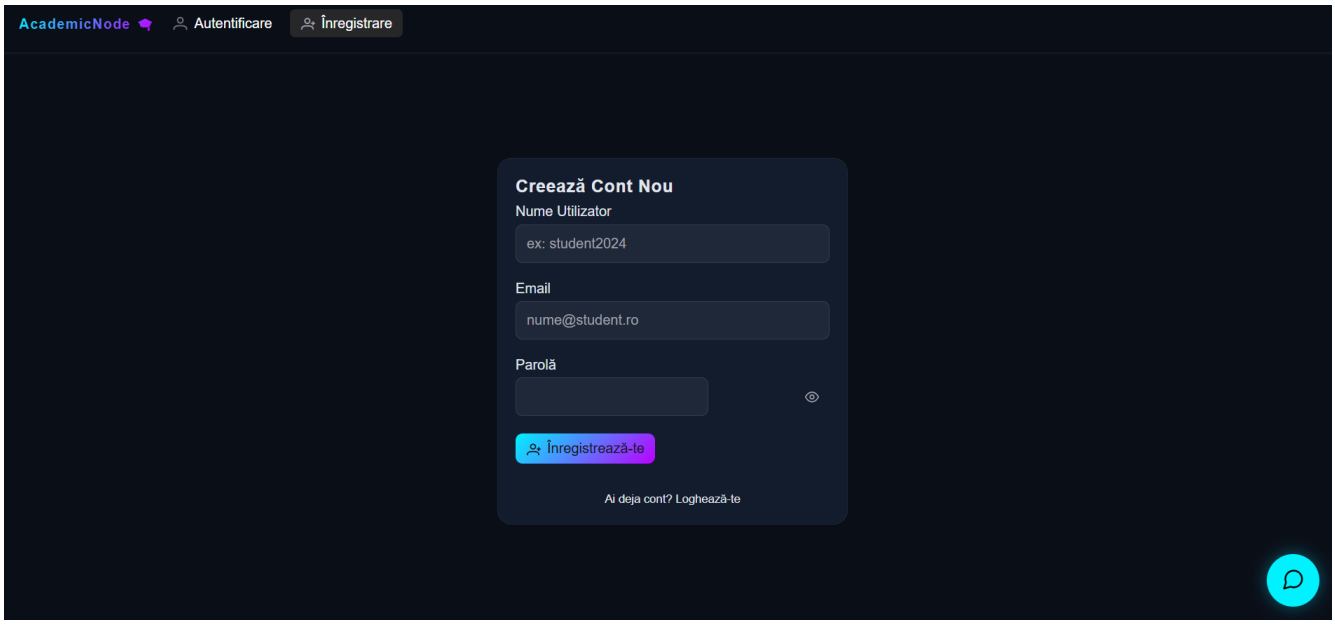
4 Application Implementation and Results

This chapter explains how the AcademicNode platform was built. It covers the main business logic, how the user interface was created, and how advanced academic features were added. The focus throughout was on keeping the code modular, protecting data, and making the platform easy to use.

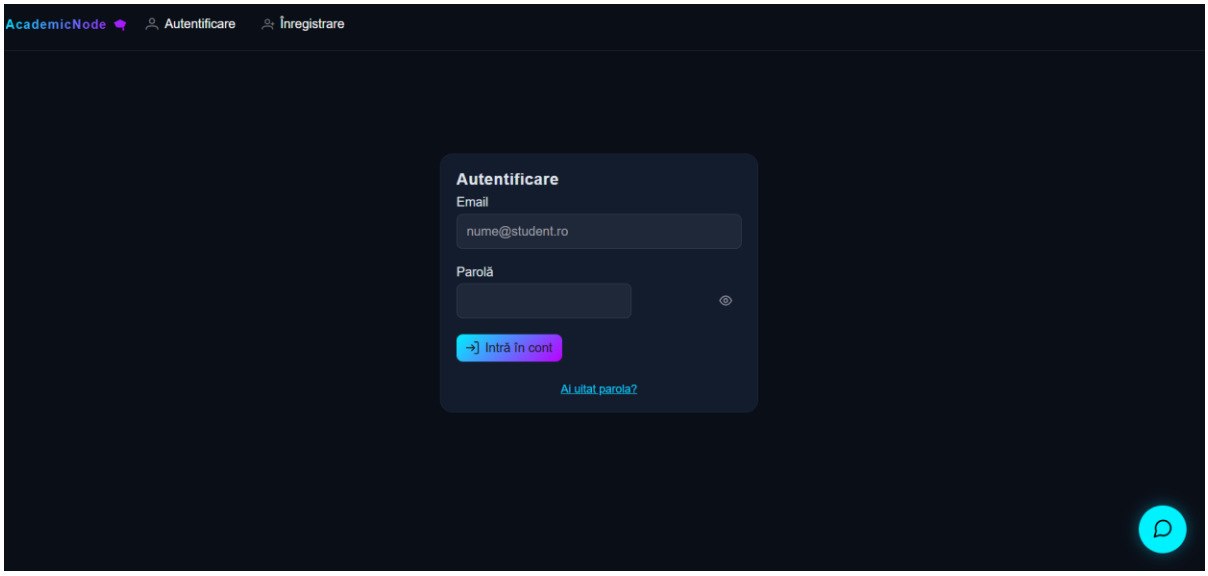
4.1. Authentication and Identity Management System

When I first started using AcademicNode, I was relieved to see how seriously it takes security. The platform uses the ASP.NET Core Identity framework to handle everything from signing up to logging in and recovering accounts. Thanks to this setup, only trusted users within the academic community can access sensitive data. Here's how the authentication process works:

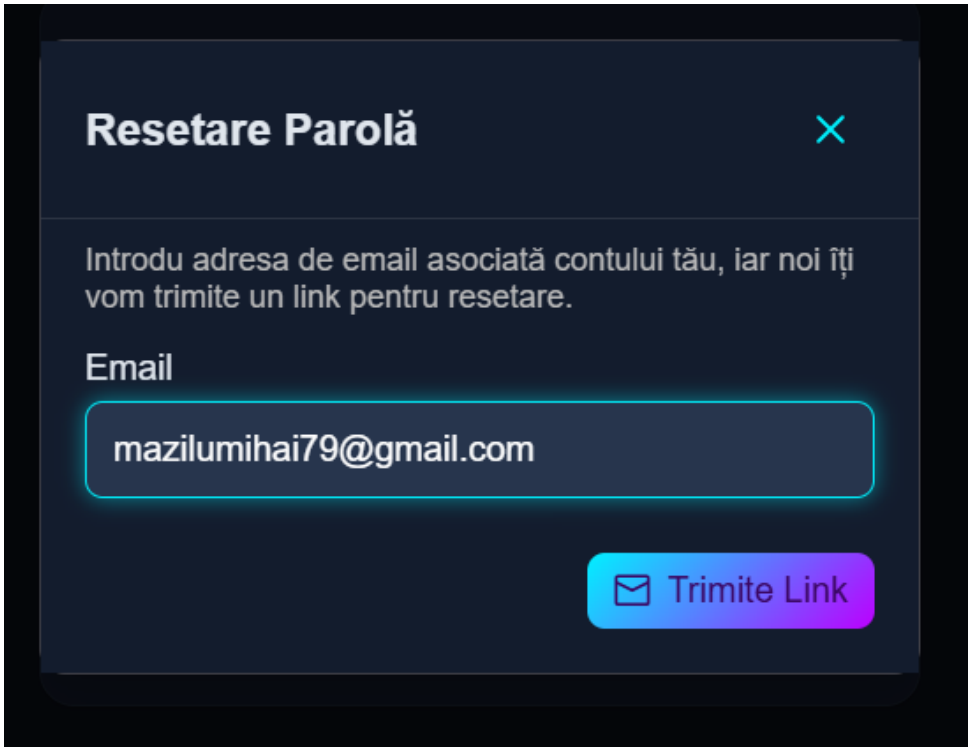
User Registration (Sign-Up): When you register, you'll be asked for a username, your academic email, and a password. The system checks your password to make sure it's strong enough—it can't be too short or too simple. Once you submit your information, the password isn't just stored as-is. Instead, it's run through a hashing process using PBKDF2 with a unique salt, which basically means even if someone got hold of the database, your password would still be safe.



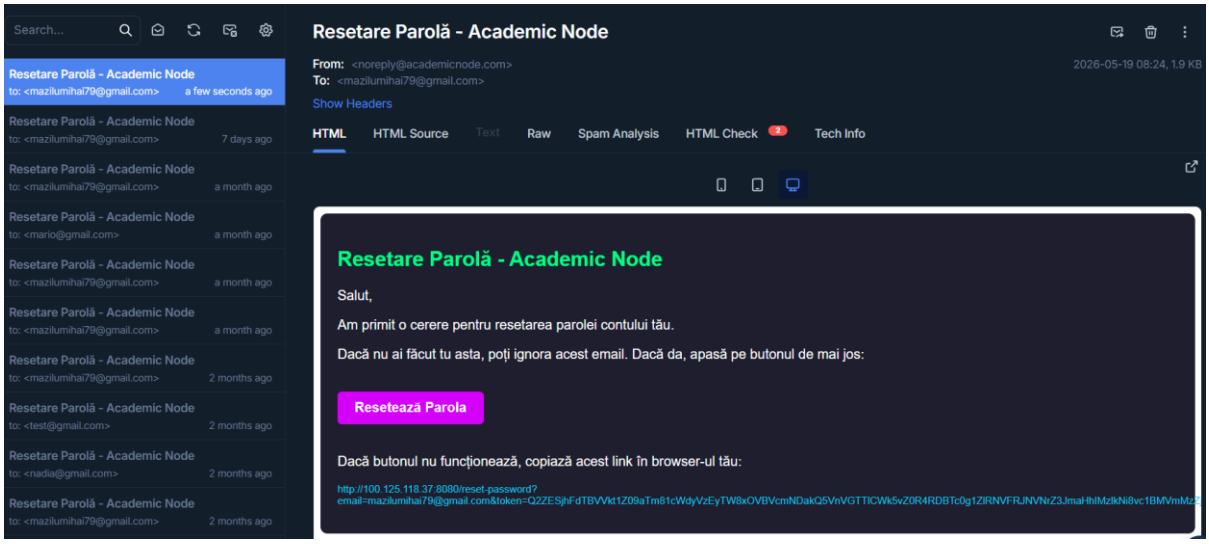
Logging In: After you've signed up, logging in is straightforward. When you enter your username and password, the system verifies them and, if they match, issues a unique digital token (a JWT). This token is like your digital ID card—it proves who you are for every action you take on the site. The best part? You don't have to keep logging in again and again, since the token handles your identity behind the scenes.



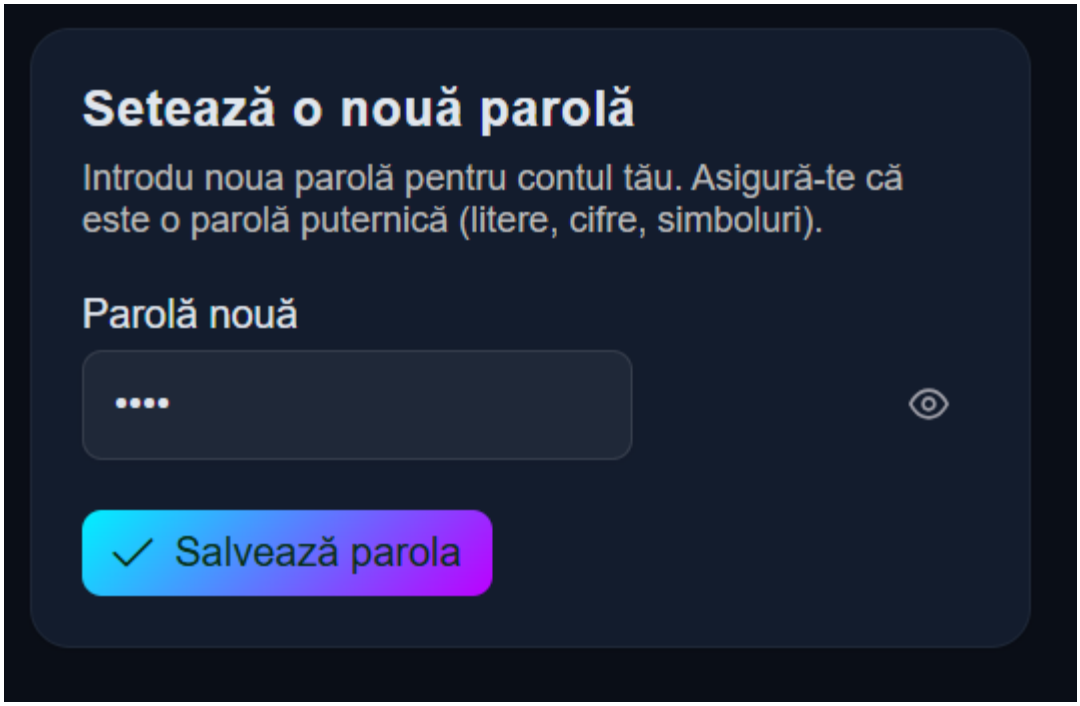
Resetting Your Password: If you forget your password, you can request a reset. AcademicNode will check your email and send you a special link. This link is only good for a short time and is tied just to you, so no one else can use it. Once you click the link and set a new password, the link expires immediately, keeping your account secure even if the email is forwarded by mistake.



MailTrap Sandbox:



Password Reset Form:

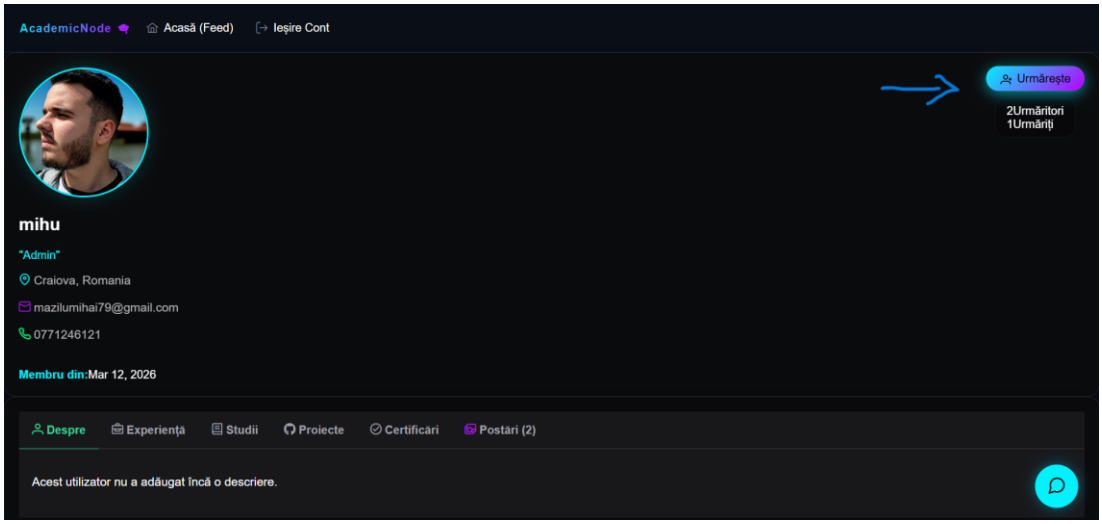


4.2. Business Logic Implementation (Follow System, Posts)

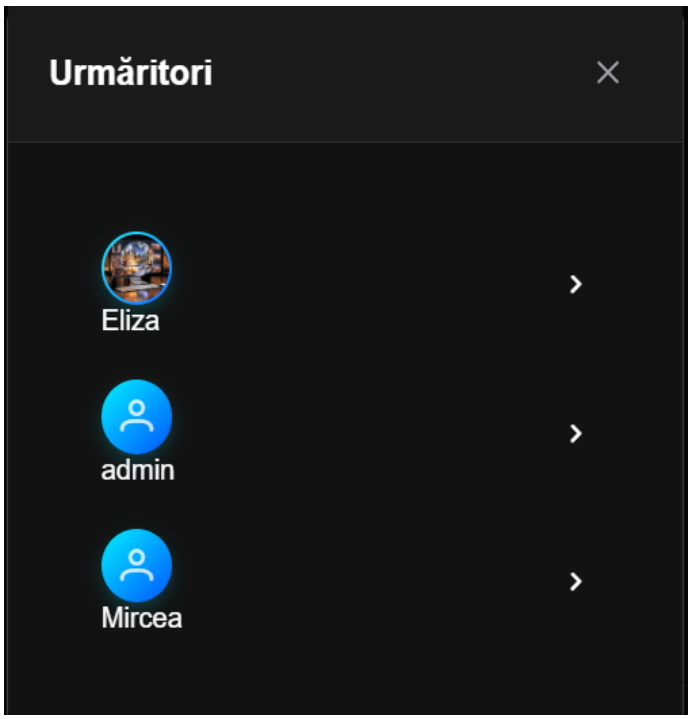
The backend is structured with controllers and services that manage complex tasks and keep communication stateless.

Follower Mechanism: The UsersController manages the follower system. When someone tries to follow another user, the system checks the relationship using the UserFollow entity. The DataContext is set up to handle self-referencing Many-to-Many relationships, which keeps data accurate. The system also blocks duplicate follows and stops users from following themselves, so the social graph stays consistent.

Follow Button:

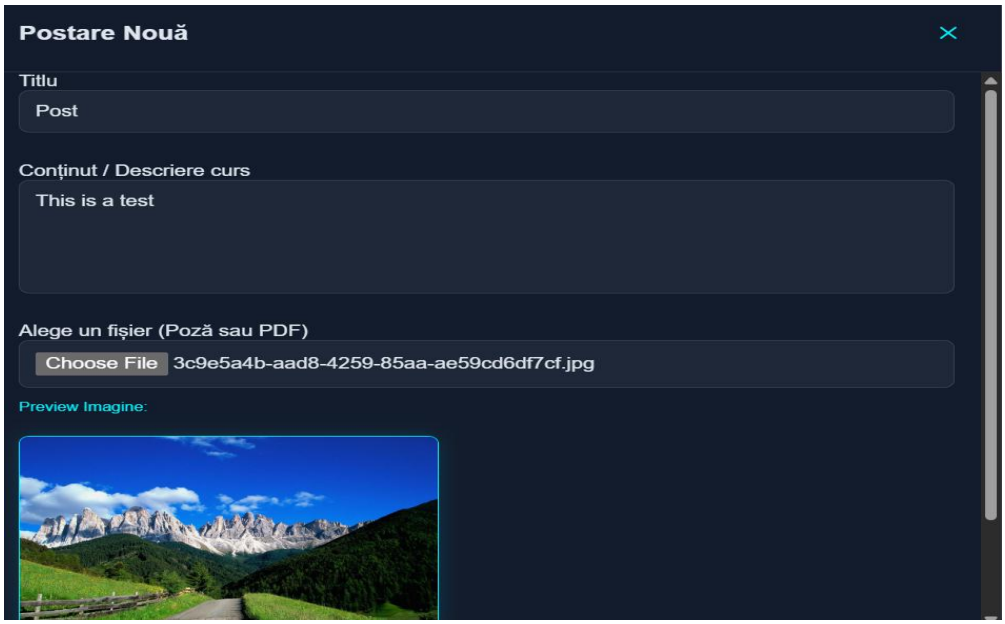


Follower List:



Post Management: The PostsController manages academic content from creation to deletion. To avoid circular reference errors when converting data to JSON, the API uses ReferenceHandler.IgnoreCycles in Program.cs. Data is sent between the server and client using special Data Transfer Objects (DTOs) like PostDto and CreatePostDto, so only the needed information is shared.

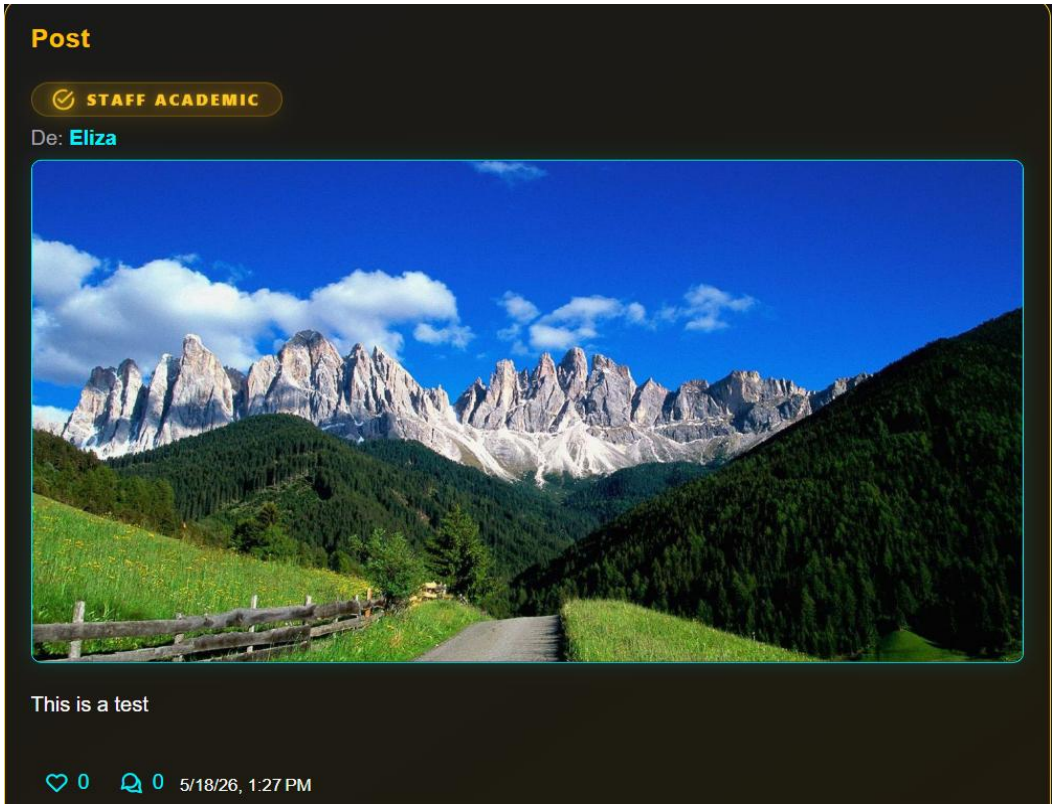
Post Container:



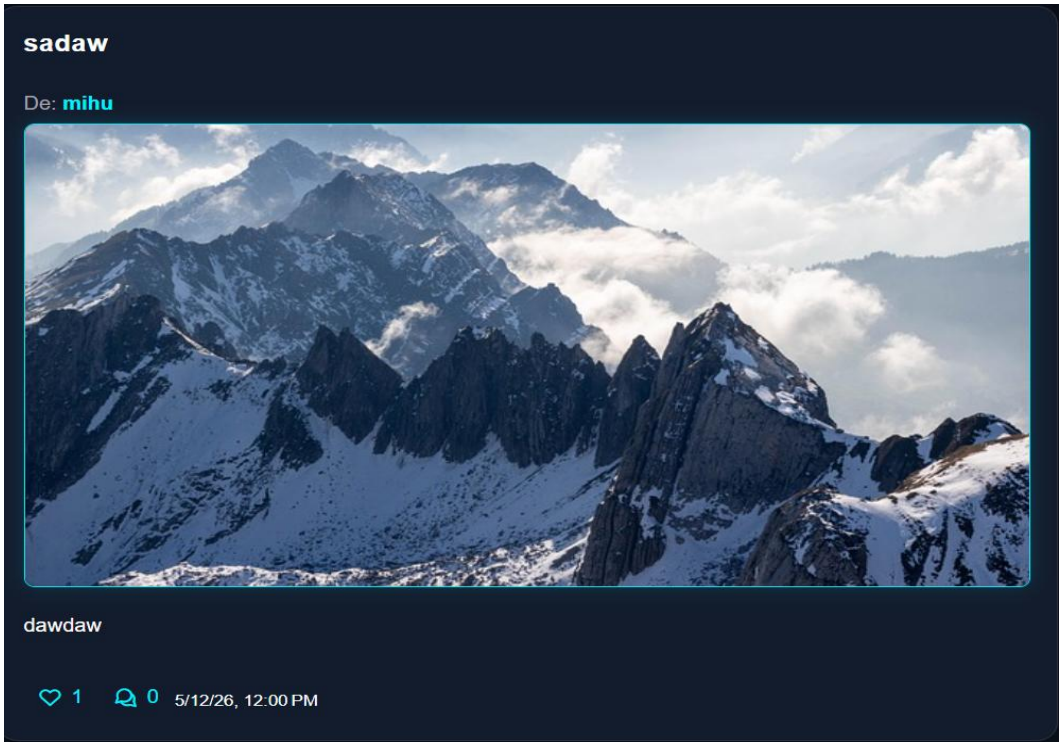
Ease of use within the Post Container interface for content creation and back-end security are top priorities. When a user enters a media file, such as a JPG image or a PDF document, the Angular front-end captures the user input (which includes any text in the Title and Description fields) using the FormData web API. This embeds the binary file and the entered text and sends it to the PostsController as a multipart upload. The back-end processes the data, but before it is saved, the request is intercepted by the FileService service in the back-end to check the file extension, MIME type, and any size restrictions. After these checks, the binary data is safely saved in the local wwwroot/documents directory. This system is implemented to prevent any malicious script injection that would try to disguise itself as media files.

Additionally, the platform's social feed feature aims to reflect the academic social hierarchy and validate the credibility of posted materials. This is achieved by using Angular's conditional UI rendering based on active identity claims. As the feed fetches posts, it parses the Role attribute against the author's DTO (Data Transfer Object). When the author has higher-level roles, such as Professor or Administrator, the app adds a unique "Staff Academic" badge and some design CSS to highlight the post container. This allows students to instantly identify and distinguish authenticated institutional announcements or official course materials from regular peer-to-peer posts, as illustrated in the following screenshots.

Professor Role Post:

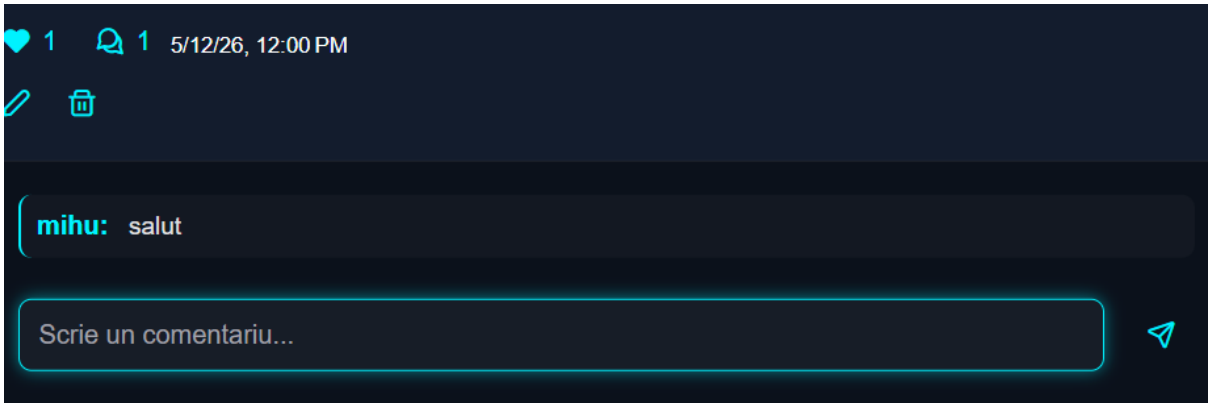


Other Roles Post:

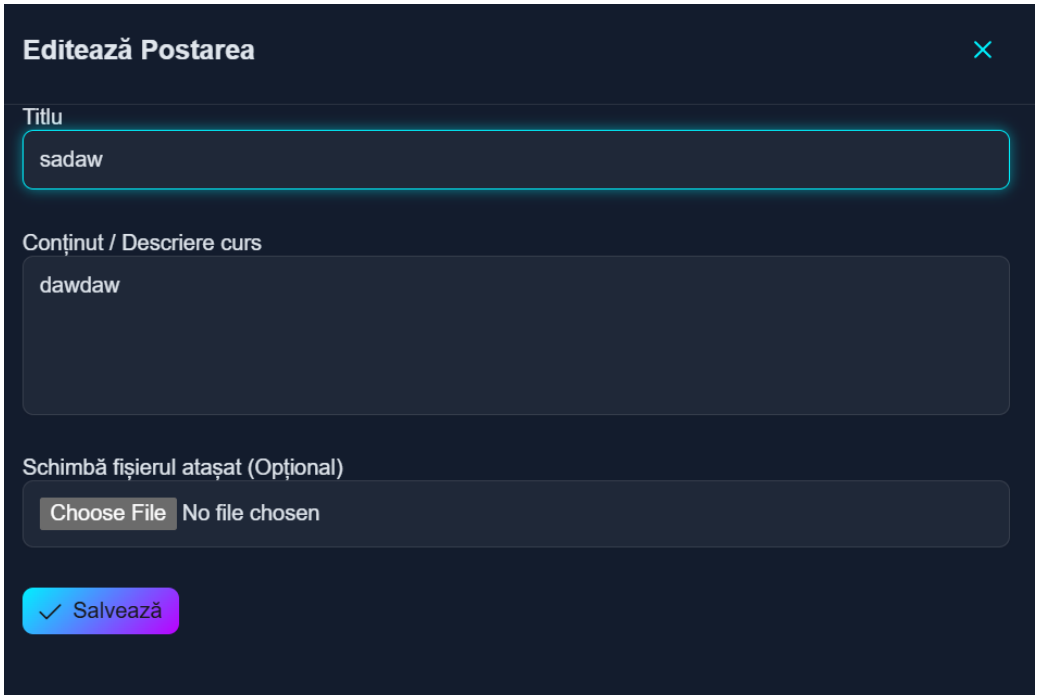


Core Post Interactions: Likes, Comments, and Content Lifecycle: Besides creating content, users can interact with posts using features managed by the backend. Logged-in users can like academic posts with a toggle button that adds or removes their like in the PostLike table. This setup makes sure each user can only like a post once.

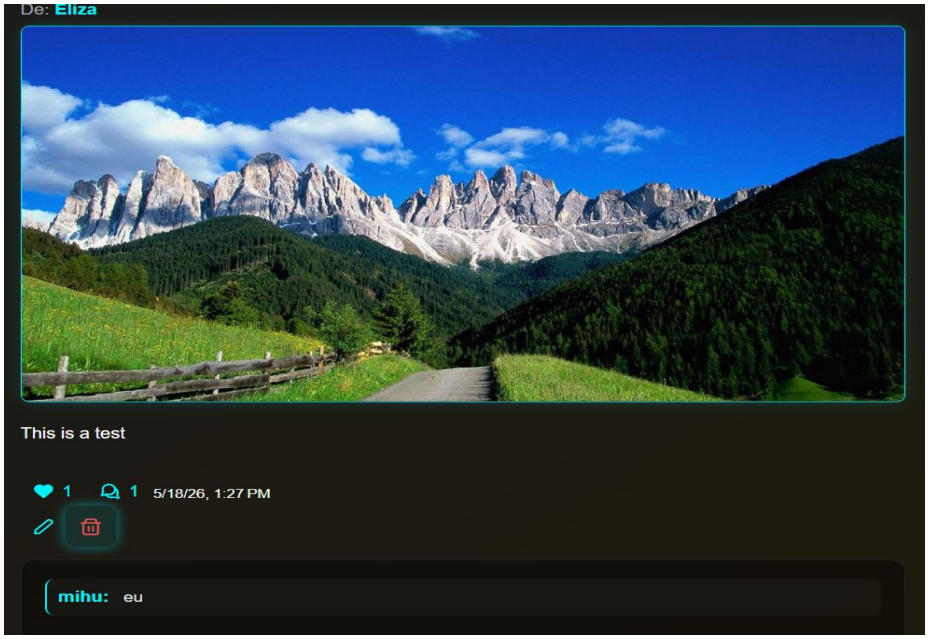
The platform also has a comment system so users can discuss posts directly. Only the original author can edit or delete their own posts. The interface only shows editing and deletion options when the logged-in user's ID matches the author's ID.



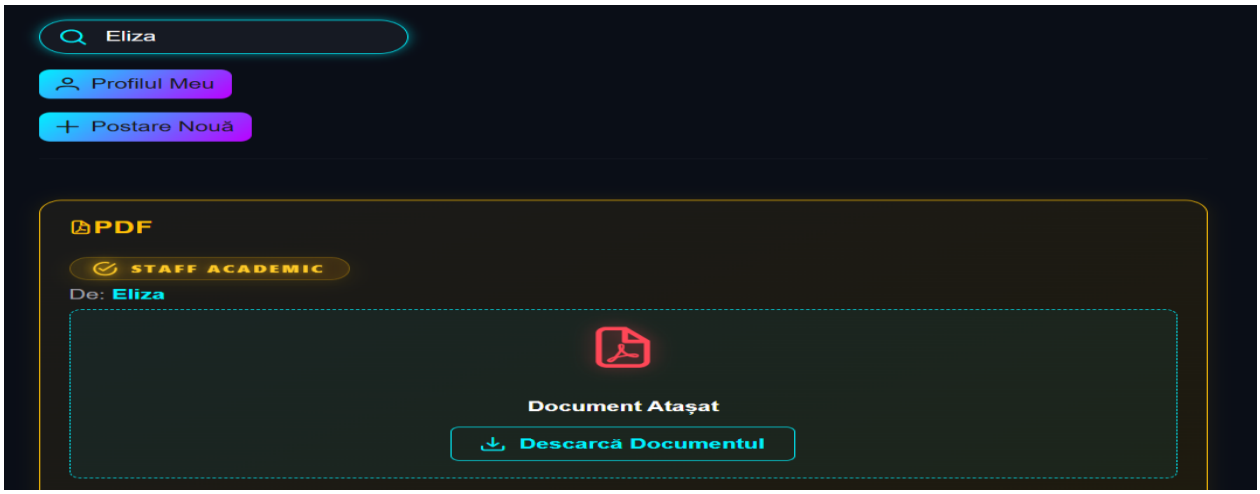
Edit Form:



Role-Based Administrative Moderation and Global Privileges: To keep academic standards and follow community rules, AcademicNode uses a Role-Based Access Control (RBAC) system to manage posts. Admin users do not have the same ownership limits as regular users. Special admin endpoints, protected by the [Authorize(Roles = "Admin")] attribute in .NET, let moderators edit or delete any post, comment, or shared resource on the platform. The interface only shows moderation controls to users with admin rights.



Content Discovery and Search Query Architecture: The platform includes a global search and filter system to help users find information easily. The search engine uses a special endpoint in the PostsController to retrieve search terms from the search bar on the frontend, then translates the user's input into optimized queries. When a user searches, the backend turns their input into LINQ queries for the database. The system searches both the post content and the author's name or username. This way, students and faculty can quickly find relevant posts or research by specific people.



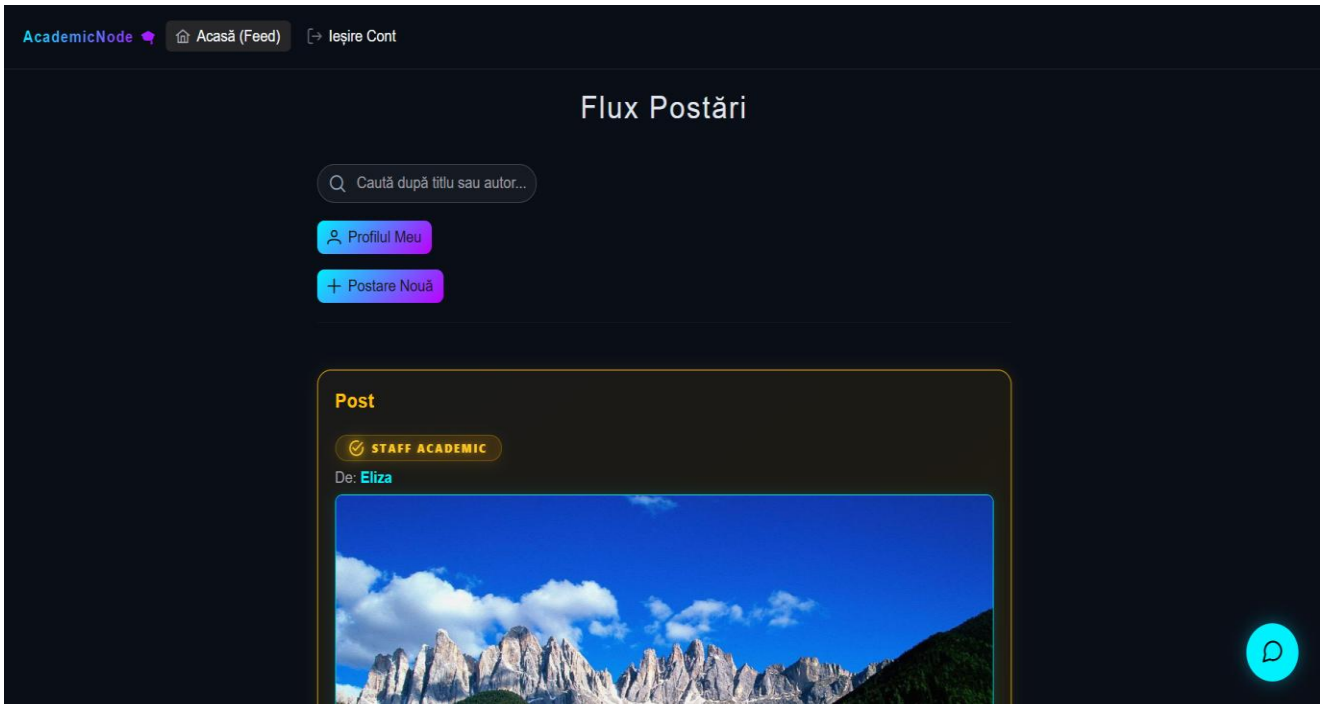
4.3. Graphic Interface and User Experience (UI/UX)

The front end uses the latest features of Angular 21 and PrimeNG 21 to build a responsive and easy-to-use interface.

Component-Based Architecture: The interface is split into modular parts like Feed, Profile, and Navbar, which makes it easier to reuse code and maintain the system. Angular's reactive forms check user input and give instant feedback during sign-up and profile updates.

PrimeNG Integration: The platform uses advanced UI components from PrimeNG, like dynamic data tables in the About section and pop-up windows for creating posts, to give a professional look. Features like Dark Mode and a responsive layout make sure the site works well on tablets, smartphones, and other devices.

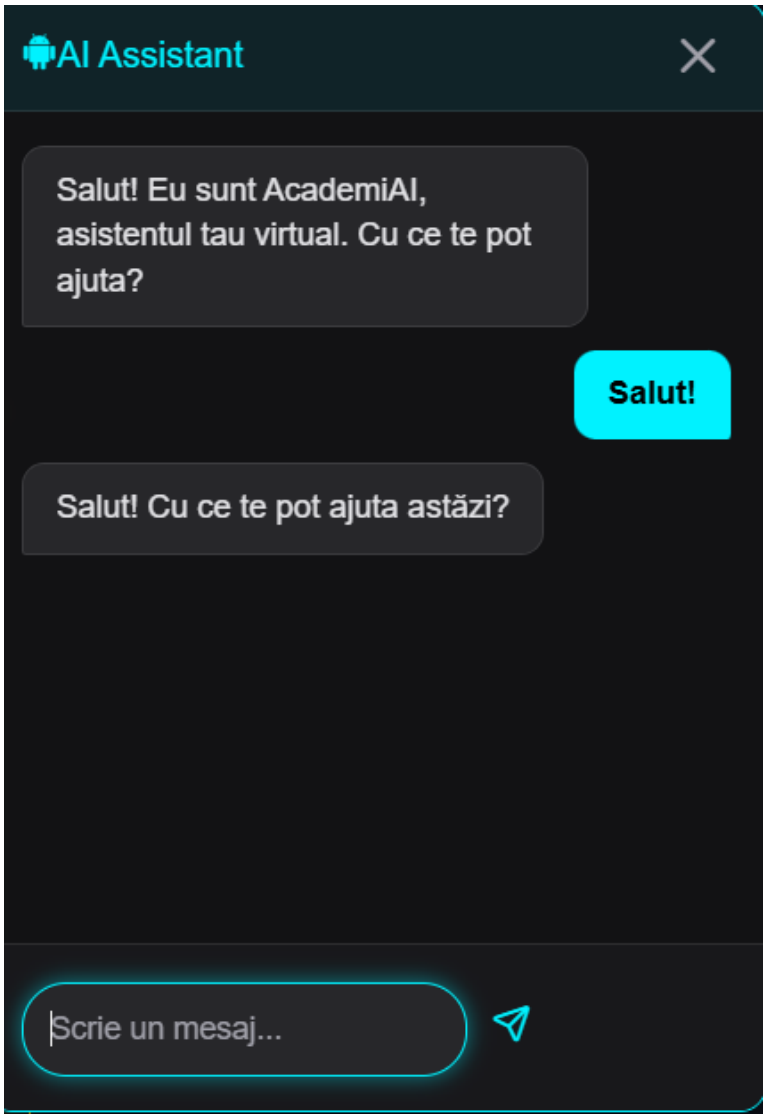
Reactive Data Streams: RxJS observables are used to fetch data in the background. This lets the app update parts of the interface, like follower counts or post likes, without reloading the whole page. As a result, the app feels faster to users.



4.4. Advanced Functionalities (AI Integration, PDF Support)

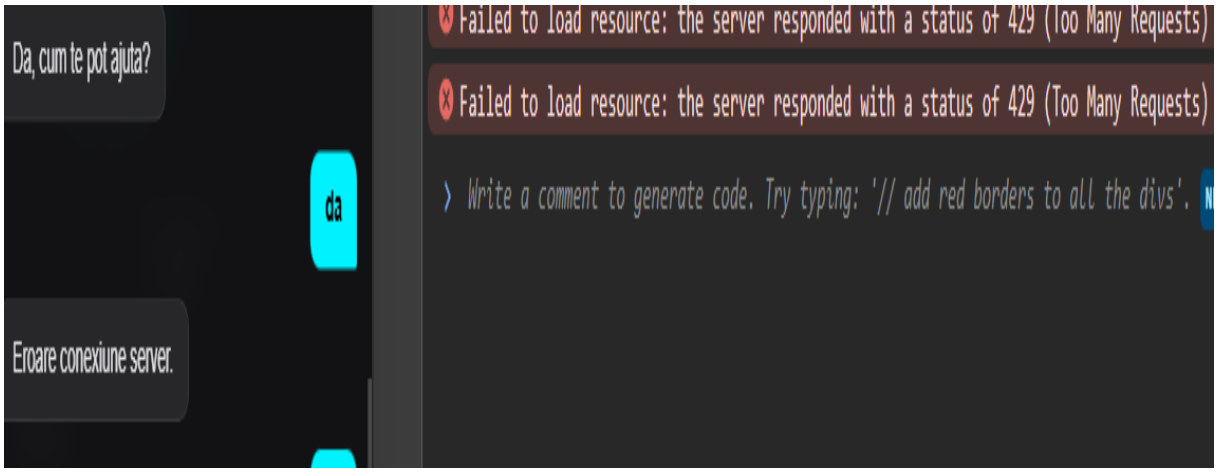
In addition to regular social features, *AcademicNode* includes special modules to help with academic tasks.

AI-Assisted Interaction: The platform includes a smart chat interface managed by the ChatController. Instead of regular user-to-user messaging, this feature lets users interact with an integrated AI assistant. It uses the ChatDto to keep data organized and secure. With this tool, users can ask academic questions, brainstorm ideas, or get automated support right on the platform.



To enhance the interactive and educational experience, *AcademicNode* integrates an Artificial Intelligence assistant powered by the Gemini Large Language Model (LLM). This feature allows users to interact with a conversational AI directly within the platform. However, the current deployment utilizes a Free Tier API key provisioned by Google, which introduces specific architectural and operational constraints.

From a system design perspective, relying on a free-tier API imposes strict rate-limiting policies (throttling) intended to prevent network abuse. These limitations manifest primarily as strict caps on Requests Per Minute (RPM) and total Requests Per Day (RPD). If concurrent user interactions spike and the threshold is exceeded, the external API will reject subsequent payloads, returning an HTTP 429 (Too Many Requests) status code. Furthermore, the free tier does not provide Enterprise-level Service Level Agreements (SLAs), meaning system availability is entirely dependent on the provider's server congestion.

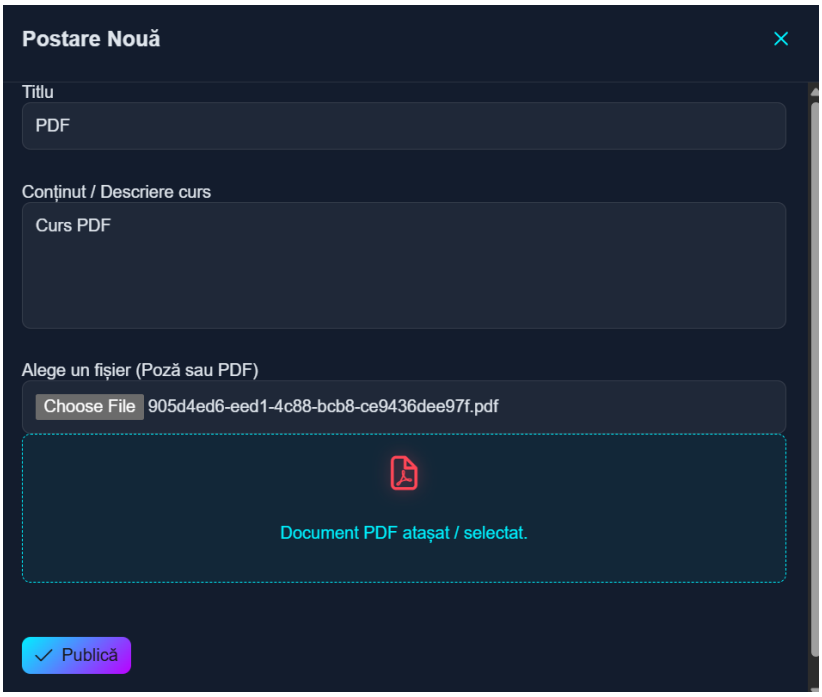


To maintain platform stability and ensure a seamless User Experience (UX), the backend and frontend architectures are designed to handle these constraints gracefully. The application intercepts rate-limit exceptions and API timeouts, preventing critical application crashes. Instead of a hard failure, the UI renders informative feedback to the user, advising them to retry their prompt later.

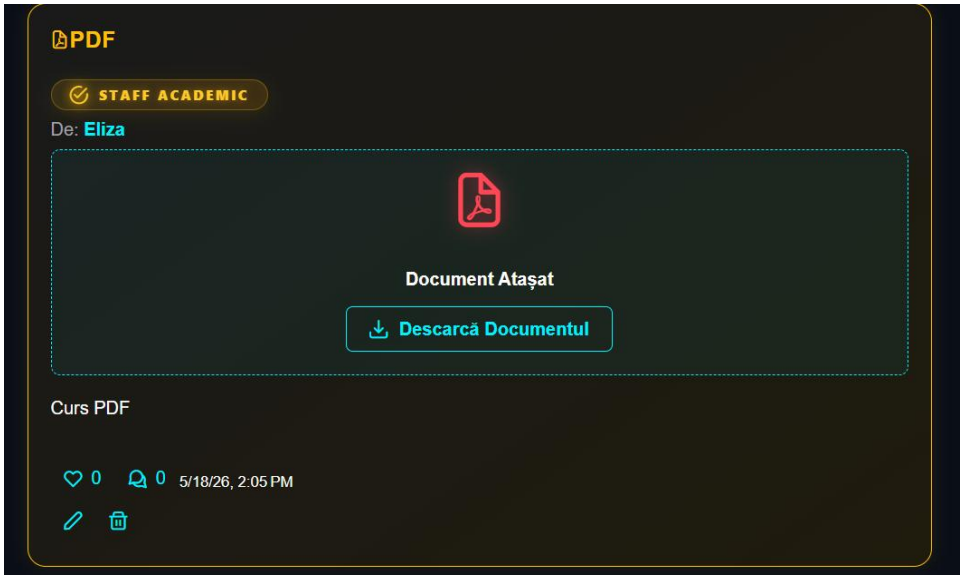
This current implementation is treated as a highly functional Proof of Concept (PoC). The system's modular architecture guarantees that for future, large-scale production environments, the free API key can be seamlessly swapped with a commercial, enterprise-grade cloud solution (such as Google Cloud Vertex AI). This upgrade path would instantly unlock dedicated quotas, extended token context windows, and guaranteed uptime, requiring absolutely no structural refactoring of the core application logic.

Academic Resource Management: The platform supports PDF documents. The FileService handles secure uploading and storage in the wwwroot/documents folder. Users can add course materials to their posts, and followers can view or download these files.

Post PDF container:

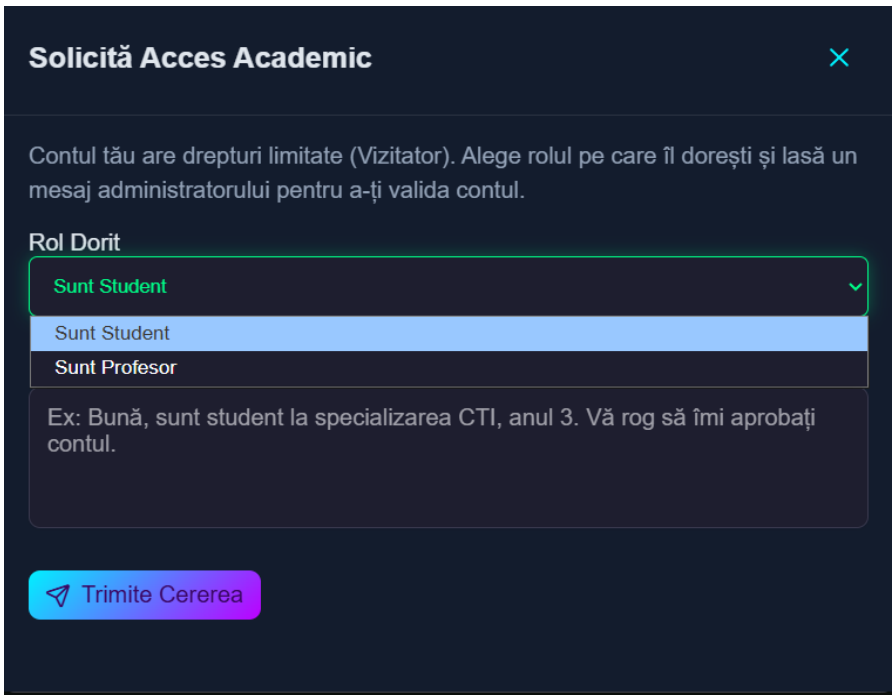


PDF Post:

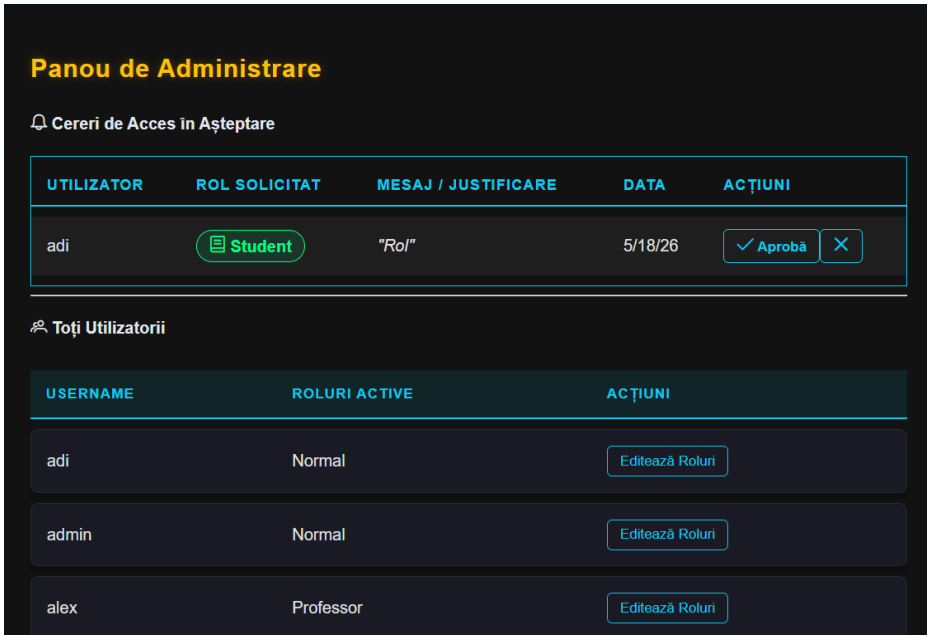


Identity and Role-Based Seeding: When the system starts, Program.cs runs a routine that creates key roles like Admin, Professor, and Student, along with a default admin account. This makes sure the platform is ready to use right after deployment. Also for a easier role management it was implemented an admin panel page where you can accept, deny or change roles of the users.

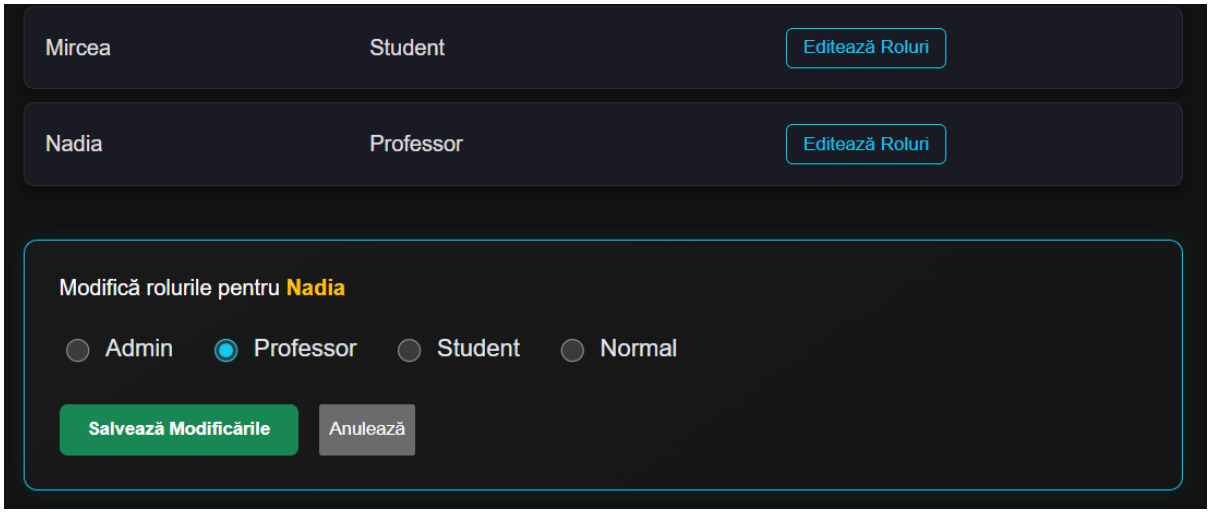
Role request by new user form:



Admin Panel:



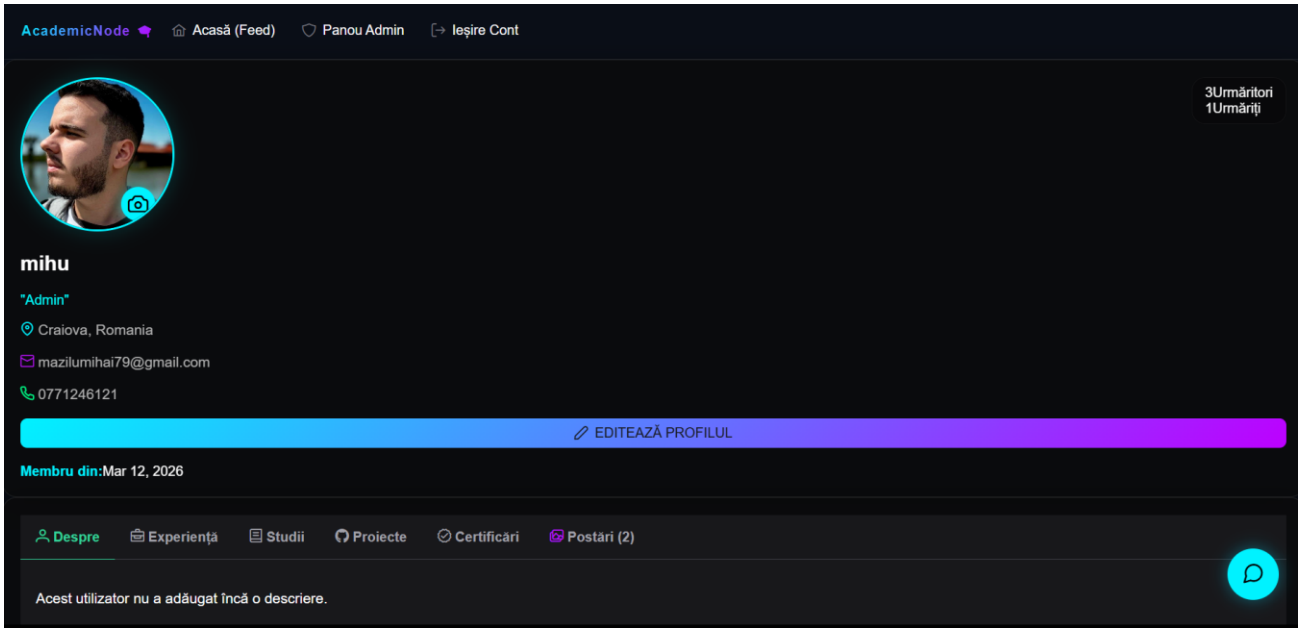
Manual changing role option:



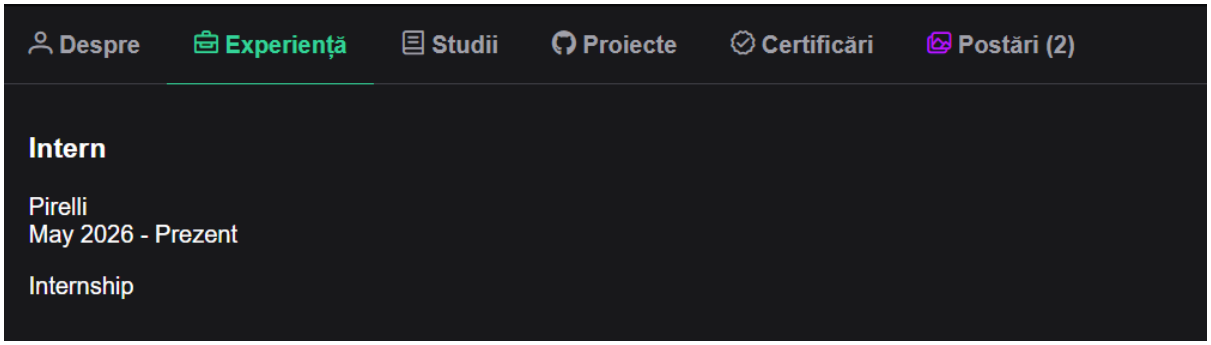
4.5. User Profile and Professional Profiling Module

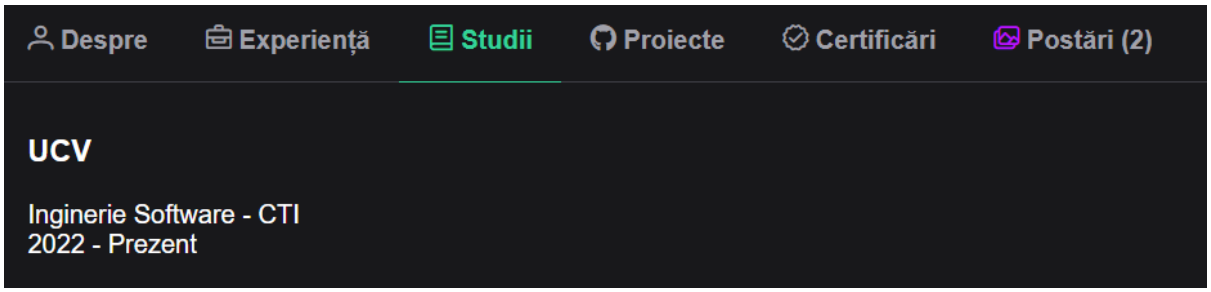
I remember the first time I filled out my AcademicNode profile—it felt a bit like gathering pieces of my academic journey in one place. The User Profile module acts as a digital CV for both students and professors, offering a snapshot of academic milestones and professional roles. This makes it much simpler to showcase achievements or reach out to others with similar interests. The module is divided into three main sections:

The "About" section collects essential details, including your city, country, academic email address, phone number and profile picture. This makes it straightforward for colleagues or collaborators to contact you if needed.



The Professional and Academic History section highlights your Education, Work Experience, Certifications, and Projects. Users can easily add or update records using simple forms, and each entry is linked to the main profile to keep information organized.

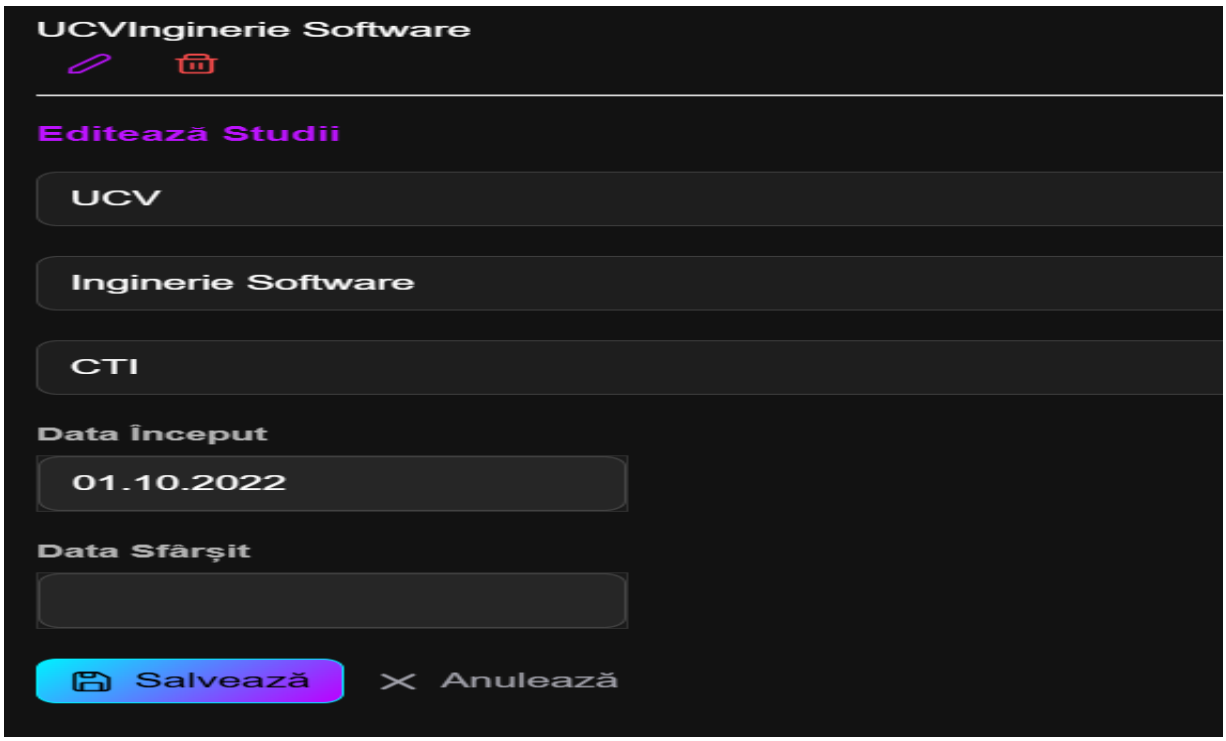




Unified Lifecycle Management of Profile Components (CRUD Operations):

To provide a fully interactive and self-managed experience, the User Profile module implements a unified architectural pattern for Create, Read, Update, and Delete (CRUD) operations across all its structural sections—including personal metadata (About), academic background (Education), career history (Experience), professional certifications, and projects. Rather than treating each section as an isolated feature, the platform utilizes a standardized data flow that bridges the Angular frontend with the secure .NET 8 Web API backend.

When a profile owner initiates a modification or appends a new record to any section, the data lifecycle follows a strict asynchronous pipeline:

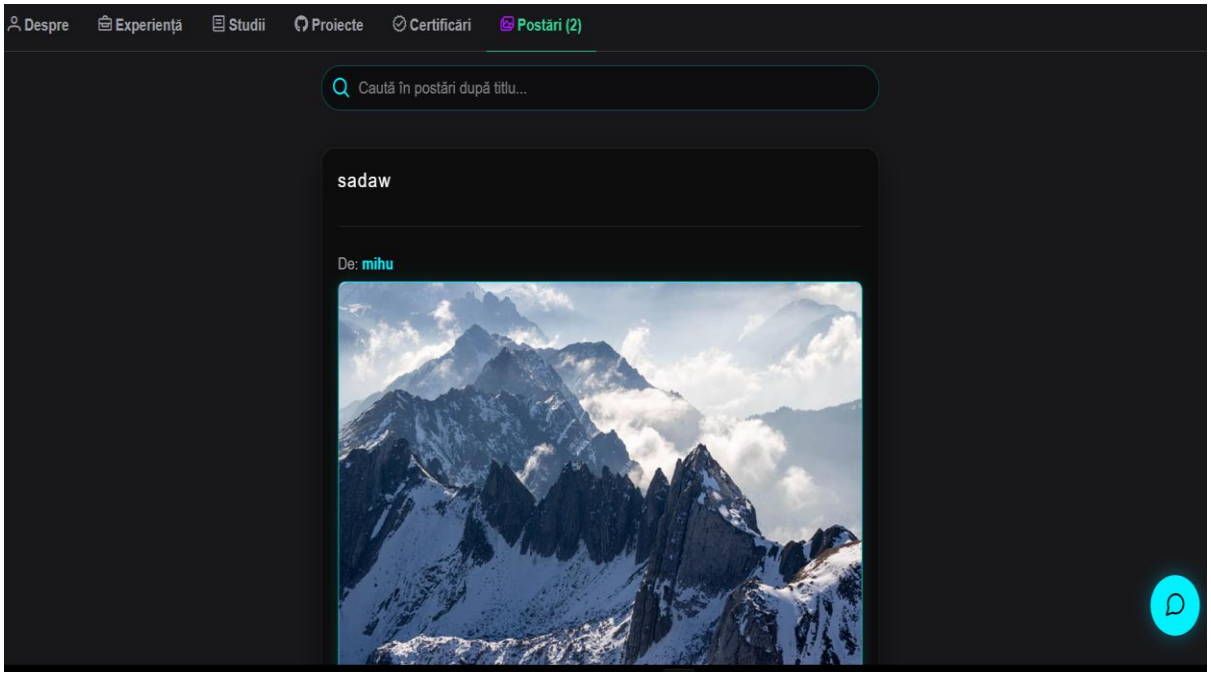


The Mutation and Edit State (Create/Update): The static display layout is dynamically replaced by a reactive Angular form embedded within a PrimeNG modal dialog, pre-populated with existing database values if an update is triggered. Client-side validators enforce strict constraints on input fields. Upon form submission, the application dispatches an optimized HTTP workload—utilizing **PUT** verbs for modifications and **POST** verbs for new entries—targeting the specific profile controller endpoints.

Backend Validation and Persistence: The server intercepts the incoming Data Transfer Objects (DTOs), performs server-side model validation, and verifies account ownership through JWT token claims to prevent unauthorized cross-profile tampering. Changes are then committed to the database context (SQL Server) using Entity Framework Core state tracking.

The Purge Mechanism (Delete): For data elimination, the user triggers a destructive action that invokes a frontend confirmation dialog to mitigate accidental loss. Upon confirmation, an HTTP **DELETE** request is transmitted. The backend safely updates or purges the corresponding entity row, ensuring relational constraints are preserved.

State Optimization (UX): To guarantee a seamless User Experience (UX), the frontend does not require a full page reload upon receiving a successful HTTP 200 OK server response. Instead, the Angular state management layer intercepts the response and instantly applies an inline filter to splice or update the local array in memory, updating the user interface in real-time.

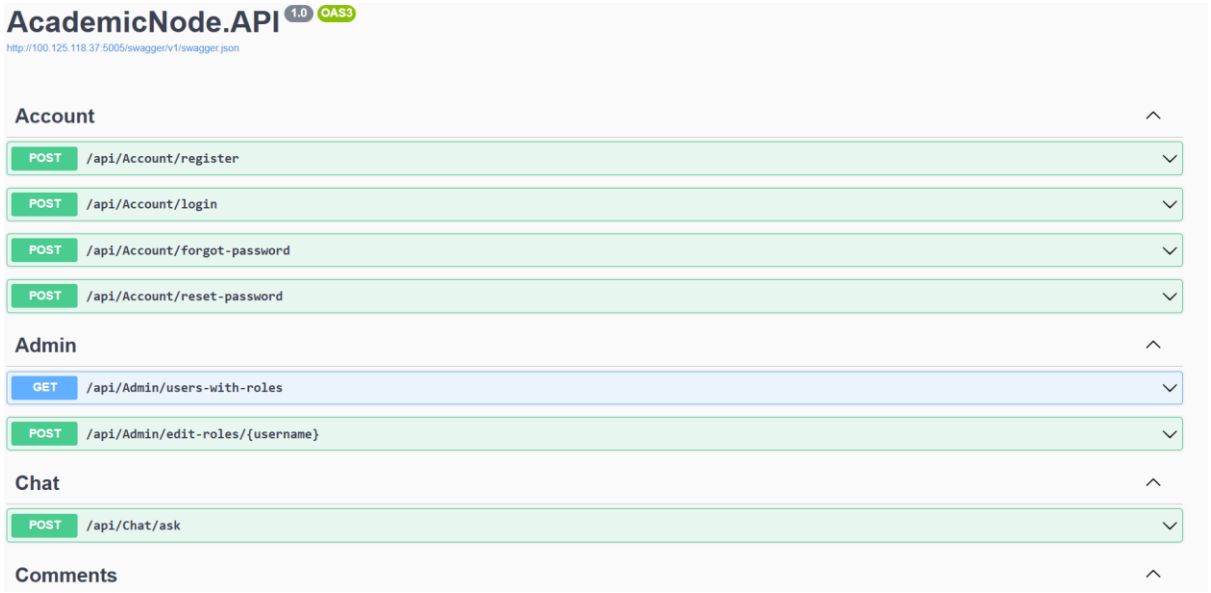


The Personal Post Feed displays only the posts and resources you've shared. This feature lets peers and faculty easily find your research, ideas, and shared documents without having to search the entire platform.

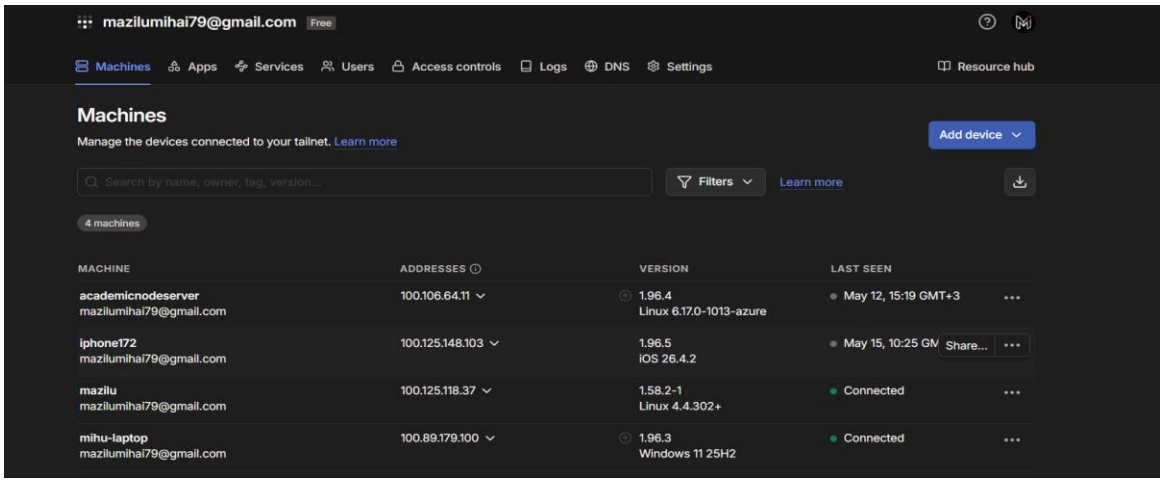
4.6. Testing and Validation

The application was thoroughly tested to make sure it works correctly and is stable.

API Testing via Swagger: During development, the **Swagger** library created an interactive Swagger UI. This made it possible to test each API endpoint separately and check that controllers handled different HTTP methods and returned the right status codes.



Deployment Validation: The containerized deployment on NAS hardware was tested by simulating heavy use. The Zero-Trust network setup was checked with Tailscale nodes to make sure the API and database could not be reached from the public internet, but authorized users could still use the system through the VPN.



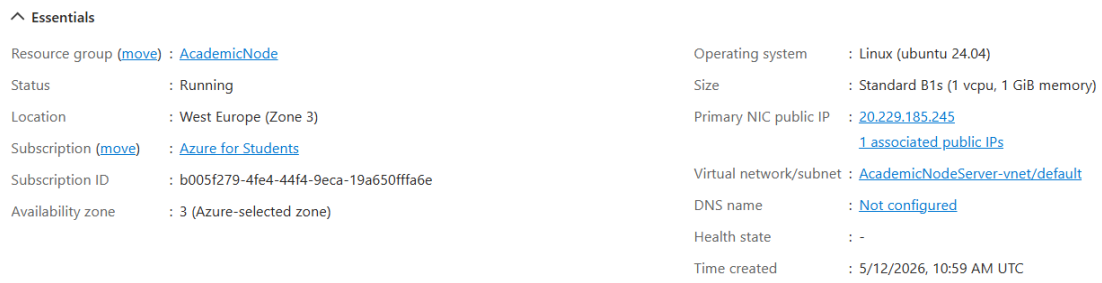
4.7. Production Deployment and Infrastructure Security

The production deployment of the **AcademicNode** platform was carried out On-Premise on a local NAS (Network Attached Storage) server, utilizing Docker technology for the containerization of the interface (Angular 17), the API (.NET 8), and the database (SQL Server). To ensure a maximum level of cybersecurity, the infrastructure was isolated through a **Zero-Trust** architecture using the Tailscale solution (Mesh VPN). This approach restricted platform access exclusively to authenticated devices, completely eliminating the vulnerabilities associated with exposing ports to the public internet.

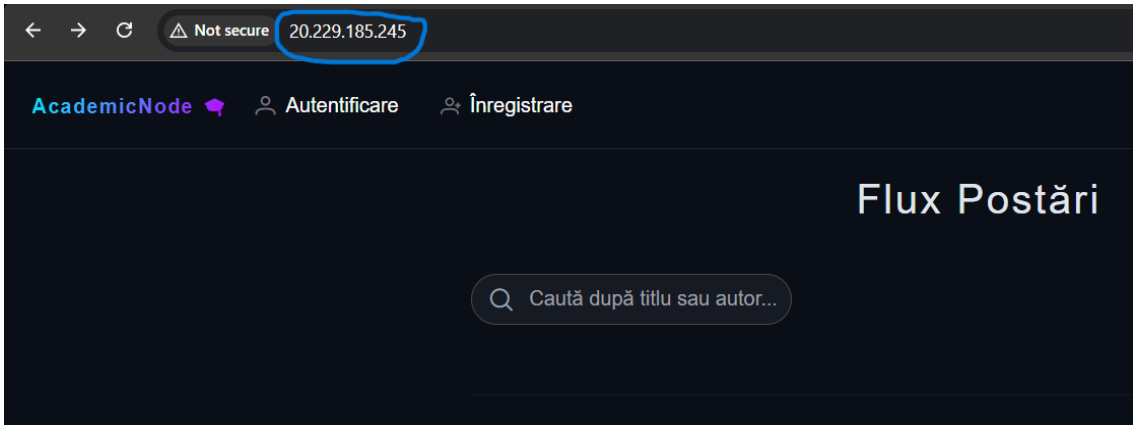
Within this secure environment, the platform underwent rigorous testing, confirming the stability of the containerized services, the proper resolution of CORS policies, and the optimal functioning of the token-based authentication mechanisms (JWT).

Additionally, a successful Hybrid-Cloud PoC scenario was tested to show that the application is flexible and scalable at an Enterprise-level. On a separate virtual machine in the Microsoft Azure environment, that was configured as Reverse Proxy (Nginx) - public traffic was allowed to pass through IPSec VPN tunnel securely from cloud to local NAS. This test confirms that the underlying architecture for the platform is "Cloud-Ready" to expose only what should be made publicly available while maintaining a strict separation of sensitive data in the local network.

Azure VM:

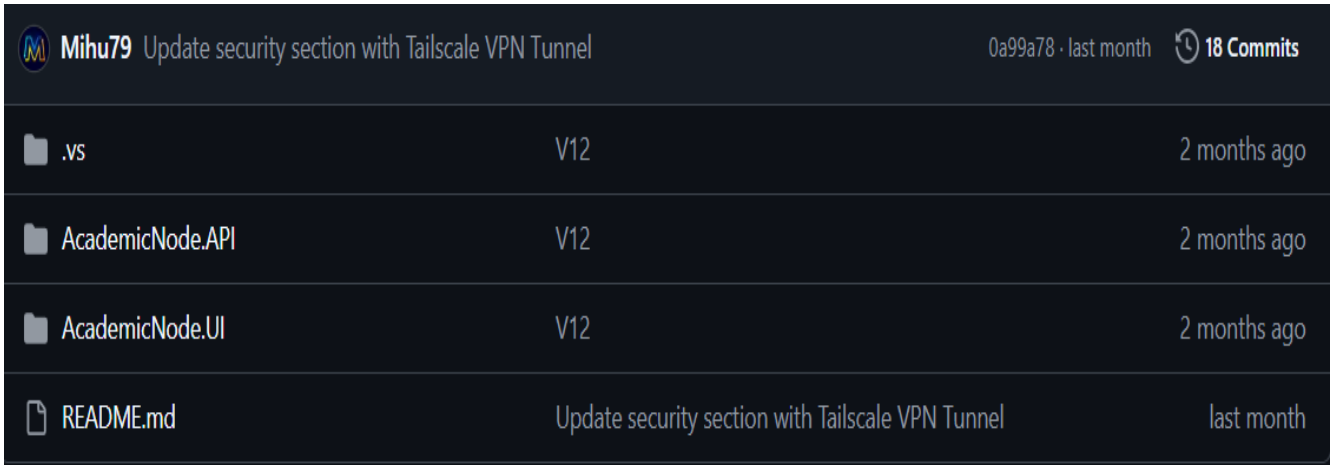


AcademicNode on Azure VM:



4.8. Version Control and Source Code Management

To ensure codebase integrity, track development milestones, and maintain a structured workflow, the *AcademicNode* platform relies on **Git** as its primary Version Control System (VCS). The entire source code, encompassing both the Angular frontend application and the .NET 8 Web API backend, is centralized and hosted on **GitHub**. This approach not only provides a reliable off-site backup but also aligns the project with industry-standard software engineering practices.



Repository Architecture and Separation of Concerns
The repository maintains a clear separation between client-side and server-side components. This structure prevents dependency conflicts and enables independent versioning and containerization for both frontend and backend modules. Each structural change, feature addition, and bug fix is documented through individual Git commits, providing a comprehensive audit trail of the platform's development.

Security Management and Data Sanitization
Hosting source code in a cloud-based repository can expose credentials by accident. To address this, strict data sanitization policies are enforced before code is pushed to GitHub. The .gitignore configuration files are used to exclude all sensitive environment variables, such as SQL Server connection strings, JWT secret keys, Gemini API free-tier keys, and Docker volume data, from version control. This ensures that only sanitized application logic is included in the repository, upholding the principle of least privilege and protecting infrastructure secrets.

5. Conclusions

This chapter reviews what was achieved in the AcademicNode project, focusing on both technical results and personal contributions. It also points out current limitations and suggests ways the project could be improved in the future.

5.1. Summary of Personal Contributions

Building AcademicNode showed how advanced software engineering methods can help solve real problems in academic collaboration. The project created a strong platform that connects traditional learning management with social networking features.
The primary technical contributions of this work include:

Full-Stack Implementation: Built a decoupled system using the latest Angular (v21) and .NET (v8), which provides a fast and responsive user experience.

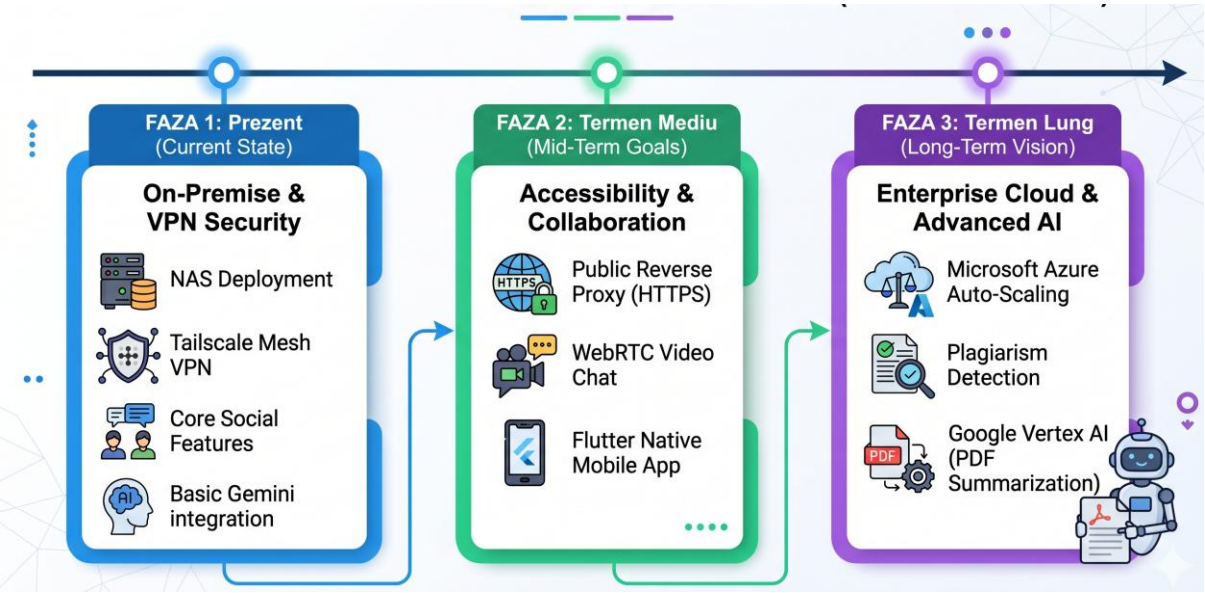
Complex Data Modeling: Designed a relational database in SQL Server that can manage complex social features like a recursive follower system and content interactions, while keeping data accurate and secure.

AI and Resource Integration: Added an AI assistant to help with academic tasks and set up a secure system for sharing PDF educational materials.

Infrastructure Innovation: Deployed the platform on a private NAS with Docker containers and used Tailscale for secure networking. This setup keeps academic data safe and only available to approved users, offering a good example of secure data management for education.

5.2. Limitations and Future Development

The platform meets its main goals, but testing and validation showed there are still areas that could be improved.



Network Accessibility: Right now, the platform uses a Mesh VPN for security, so users need to install the Tailscale client. In the future, adding a secure public reverse proxy with automatic SSL/TLS certificates could make access easier without reducing security

Real-Time Collaboration: While the AI chat is functional, adding real-time video conferencing and collaborative document editing (similar to Google Docs) would significantly enhance the platform's utility for study groups.

Mobile Integration: Creating a native mobile app for iOS and Android with tools like Ionic or Flutter would help users stay engaged and access the platform from anywhere.

Advanced AI Features: Future updates could let the AI summarize uploaded PDFs, check for plagiarism, and suggest academic content based on each user's research interests.

6 BIBLIOGRAFIE

I. Ben-Gan. (2023) T-SQL Fundamentals (4th Edition). Microsoft Press.

J. Biron and J. Sangster. (2023) Docker for Developers: Develop and run your application with Docker containers. Packt Publishing.

B. Cherny. (2019) Programming TypeScript: Making Your JavaScript Applications Scale. O'Reilly Media.

S. Clarke. (2022) Pro ASP.NET Core Identity: Under the Hood with Authentication and Authorization. Apress.

P. Deitel and H. Deitel. (2023) Internet & World Wide Web How to Program. Pearson.

J. A. Donenfeld. (2017) WireGuard: Next Generation Kernel Network Tunnel. Network and Distributed System Security Symposium (NDSS).

R. T. Fielding. (2000) Architectural Styles and the Design of Network-based Software Architectures. Ph.D. Dissertation, University of California, Irvine.

A. Freeman. (2024) Pro Angular: Build Powerful and Dynamic Web Apps. Apress.

Google DeepMind. (2023) Gemini: A Family of Highly Capable Multimodal Models. arXiv preprint arXiv:2312.11805.

L. Hunter. (2021) Reactive Programming with RxJS 7. Packt Publishing.

M. Jones, J. Bradley, and N. Sakimura. (2015) JSON Web Token (JWT). RFC 7519, Internet Engineering Task Force (IETF).

Microsoft Corporation. (2024) C# 12 and .NET 8 – Modern Cross-Platform Development Fundamentals. Microsoft Press.

Microsoft Corporation. (2024) Entity Framework Core in Action. Manning Publications.

OWASP Foundation. (2024) OWASP Top 10 Web Application Security Risks. Open Worldwide Application Security Project.

S. Rose, O. Borchert, S. Mitchell, and S. Connelly. (2020) Zero Trust Architecture. NIST Special Publication 800-207, National Institute of Standards and Technology.

7 REFERINȚE WEB

Auth0. (2026) JWT.io - JSON Web Tokens Introduction, accesat: 2026-05-12. [Online]. Available: https://jwt.io/introduction

Docker Inc. (2026) Docker Official Documentation, accesat: 2026-05-12. [Online]. Available: https://docs.docker.com/

GitHub Inc. (2026) GitHub Docs - Version Control and Collaboration, accesat: 2026-05-12. [Online]. Available: https://docs.github.com/

Google. (2026) Gemini API Official Documentation, accesat: 2026-05-12. [Online]. Available: https://ai.google.dev/docs

Google / Angular Team. (2026) Angular Official Documentation (v21), accesat: 2026-05-12. [Online]. Available: https://angular.dev/

Microsoft Corporation. (2026) .NET 8 Documentation, accesat: 2026-05-12. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/

Microsoft Corporation. (2026) ASP.NET Core Identity Documentation, accesat: 2026-05-12. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity

Microsoft Corporation. (2026) Entity Framework Core Documentation, accesat: 2026-05-12. [Online]. Available: https://learn.microsoft.com/en-us/ef/core/

Microsoft Corporation. (2026) SQL Server Technical Documentation, accesat: 2026-05-12. [Online]. Available: https://learn.microsoft.com/en-us/sql/sql-server/

PrimeTek. (2026) PrimeNG - The Most Complete UI Suite for Angular, accesat: 2026-05-12. [Online]. Available: https://primeng.org/

ReactiveX. (2026) RxJS - Reactive Extensions Library for JavaScript, accesat: 2026-05-12. [Online]. Available: https://rxjs.dev/

SmartBear Software. (2026) Swagger UI Documentation, accesat: 2026-05-12. [Online]. Available: https://swagger.io/tools/swagger-ui/

Tailscale Inc. (2026) Tailscale - Zero config VPN for building secure networks, accesat: 2026-05-12. [Online]. Available: https://tailscale.com/

A. CODUL SURSĂ

Întregul cod sursă dezvoltat pentru platforma *AcademicNode*, incluzând atât componenta de interfață grafică (Frontend), cât și logica de server (Backend), este versionat și găzduit pe platforma GitHub.

Pentru a asigura conformitatea cu bunele practici de securitate discutate în lucrare, fișierele de configurare care conțin date sensibile (cum ar fi cheia API gratuită pentru Gemini, string-urile de conexiune la baza de date SQL Server și secretele criptografice pentru token-urile JWT) au fost excluse în mod intenționat din repository prin intermediul directivelor .gitignore.

Codul sursă poate fi accesat, consultat și clonat la următoarea adresă web: https://github.com/Mihu79/AcademicNode.git

Structura principală a repository-ului:

AcademicNode.UI: Conține proiectul de tip Single Page Application dezvoltat în Angular, alături de componentele vizuale PrimeNG și logica de management al stărilor.

AcademicNode.API: Conține soluția .NET 8 Web API, configurările Entity Framework Core (modelele și migrările Code-First), controllerele de rețea și logica de integrare a asistentului AI.

B. SITE-UL WEB AL PROIECTULUI

Datorită arhitecturii de securitate *Zero-Trust* implementate la nivel de infrastructură, platforma *AcademicNode* nu este expusă în mod intenționat pe internetul public. Această decizie arhitecturală a fost luată pentru a proteja integritatea datelor, evitând vulnerabilitățile specifice expunerii porturilor publice și prevenind accesul neautorizat la resursele academice.

Aplicația este găzduită pe infrastructura privată (NAS) și poate fi accesată exclusiv din interiorul rețelei virtuale securizate (Mesh VPN). Pentru evaluarea practică a funcționalităților, platforma poate fi accesată prin intermediul rețelei Tailscale a administratorului (owner-ului), la următoarea adresă privată:

Adresă Web (Tailscale IP): http://100.125.118.37:8080/

Notă tehnică privind accesul:

Adresa IP din clasa 100.x.x.x este o adresă CGNAT (Carrier-Grade NAT) alocată automat de protocolul de rutare Tailscale.

Portul 8080 reprezintă punctul de expunere al containerului Docker care servește interfața grafică (Frontend-ul) către rețeaua virtuală.

Condiție de acces: Încărcarea acestui link necesită ca dispozitivul client (laptop sau smartphone) să aibă instalat agentul Tailscale și să fie aprobat ca nod valid (autentificat) în rețeaua privată a proiectului.

C. CD / DVD / STICK USB

Pe suportul media extern (Stick USB) atașat fizic prezentei lucrări se regăsesc toate resursele tehnice necesare pentru evaluarea, configurarea și analizarea proiectului *AcademicNode*:

Cod Sursa Conține codul sursă integral al platformei, structurat în subdirectoarele aferente componentei de Client și componentei de Server.

Documentatie Include versiunea digitală completă a prezentei lucrări de licență în format PDF, garantând trasabilitatea documentației tehnice.

Diagrama Arhitecturii Generale (interacțiunea Client-Server).

Diagrama de Deployment (infrastructura Docker și Zero-Trust VPN).

Diagrama Entitate-Relație (ERD - Entity-Relationship Diagram) ce ilustrează schema bazei de date SQL.

INDEX